

北京理工大学

本科生毕业设计（论文）

面向代码变更的多智能体回归测试用例生成
方法

Multi-agent Regression Test Case Generation Method for Code
Changes

学 院： 计算机学院

专 业： 人工智能

班 级： 07162202

学生姓名： 王穆祺

学 号： 1120222608

指导教师： 赵三元

2026 年 5 月 12 日

原创性声明

特此申明。

本人签名: _____ 日期: _____ 年 _____ 月 _____ 日

关于使用授权的声明

本人签名: _____ 日期: _____ 年 _____ 月 _____ 日

指导老师签名: _____ 日期: _____ 年 _____ 月 _____ 日

面向代码变更的多智能体回归测试用例生成方法

摘 要

随着软件系统规模与复杂度持续增长，软件质量保障面临严峻挑战。传统软件测试高度依赖人工经验，存在效率低、覆盖不足、成本高等问题。如何提高测试效率、保证测试质量，已成为软件工程领域的重要研究问题。

近年来，敏捷开发与持续集成模式得到广泛应用，代码变更更加频繁，回归测试成为保证软件质量的关键环节。传统的回归测试主要依赖人工编写测试用例，难以快速响应频繁的代码变更。

代码变更影响分析是回归测试的核心环节，其目标在于识别代码变更可能影响的系统范围，从而指导测试资源的分配。传统方法仅能识别行级变化，难以反映代码的真实语义与影响范围。

大语言模型在代码理解、自然语言处理等方面展现出强大能力，多智能体系统通过多个 **Agent** 的协同决策，能够处理复杂多步骤的任务。本研究将大语言模型与多智能体系统相结合，提出基于多智能体的软件测试自动化方法，实现从代码变更到测试用例生成的端到端自动化。

本研究的主要贡献包括：（1）提出基于结构化中间表示的代码变更影响分析方法；（2）设计多智能体协同测试生成框架；（3）构建规则约束与 LLM 生成相结合的混合测试策略；（4）实现基于 **TestRepairAgent** 的测试修复机制。

关键词：北京理工大学；本科生；毕业设计（论文）——请在“**main.tex**”开头设置

Multi-agent Regression Test Case Generation Method for Code Changes

Abstract

With the continuous growth of software system scale and complexity, software quality assurance faces severe challenges. Traditional software testing heavily relies on manual experience, resulting in low efficiency, insufficient coverage, and high costs.

In recent years, agile development and continuous integration have been widely adopted, making code changes more frequent and regression testing critical for software quality. Traditional regression testing mainly depends on manually 编写测试用例, making it difficult to respond quickly to frequent code changes.

Change impact analysis is a core component of regression testing, aiming to identify the system scope affected by code changes. Traditional methods can only identify line-level changes and fail to reflect the true semantics and impact scope.

Large language models demonstrate strong capabilities in code understanding and natural language processing. Multi-agent systems can handle complex multi-step tasks through collaborative decision-making. This research combines large language models with multi-agent systems to propose an automated software testing method, achieving end-to-end automation from code changes to test case generation.

Key Words: BIT; Undergraduate; Graduation Project (Thesis)

目 录

摘 要	I
Abstract	II
第 1 章 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	2
1.2.1 软件测试自动化研究现状	2
1.2.2 代码变更影响分析研究	3
1.2.3 多智能体与大模型在软件工程中的应用	4
1.3 本文主要研究内容	5
1.4 本文组织架构	6
第 2 章 相关技术与方法	7
2.1 软件测试相关技术	7
2.1.1 单元测试与集成测试	7
2.1.2 自动化测试框架（Pytest）	7
2.2 代码变更分析技术	8
2.2.1 Unified Diff 解析	8
2.2.2 代码变更图与依赖关系建模	8
2.2.3 代码语义表示方法（AST 与 LLM）	9
2.3 多智能体系统技术	10
2.3.1 基于 LangGraph 的工作流建模	10
2.3.2 多 Agent 协同机制	10
2.4 大语言模型技术	11
2.4.1 代码理解与结构化生成	11
2.4.2 测试用例生成与修复	11
第 3 章 基于代码变更的影响分析方法	12
3.1 问题定义与总体架构	12
3.1.1 问题定义	12
3.1.2 核心理念：结构化中间表示	12
3.1.3 总体架构设计	13
3.2 代码变更信息提取	14
3.2.1 Unified Diff 解析与结构化	14

3.2.2 变更文件与代码块建模	14
3.2.3 变更实体抽取	15
3.3 Change Graph 构建	16
3.3.1 代码依赖关系建模	16
3.3.2 Change Graph 构建方法	16
3.4 语义单元建模方法	16
3.4.1 原子语义单元定义	17
3.4.2 语义标签体系	17
3.5 影响传播与风险评估模型	18
3.5.1 跨符号影响传播机制	18
3.5.2 影响范围限定策略	18
3.5.3 风险评分与优先级量化	19
3.6 影响分析结果生成与结构化输出	19
3.6.1 结构化输出设计	19
3.6.2 与下游模块的接口规范	20
第 4 章 多智能体测试生成系统设计与实现	21
4.1 系统总体架构设计	21
4.1.1 系统总体架构	21
4.1.2 多智能体协同流程	22
4.1.3 质量门控机制	24
4.2 核心 Agent 模块设计	24
4.2.1 变更理解阶段	24
4.2.2 测试规划阶段	26
4.2.3 测试生成阶段	27
4.2.4 测试执行阶段	28
4.2.5 评估反馈阶段	29
4.3 测试生成策略	29
4.3.1 基于变更的测试策略	30
4.3.2 结构化中间表示传递	30
第 5 章 系统测试与实验分析	32
5.1 实验环境与数据集	32
5.1.1 实验环境配置	32
5.1.2 实验数据集	32

5.1.3 Baseline 工具	32
5.2 评价指标	33
5.2.1 测试覆盖率指标	33
5.2.2 测试质量指标	33
5.2.3 测试效率指标	34
5.2.4 测试选择效果指标	34
5.3 对比实验结果分析	34
5.3.1 测试覆盖率对比	34
5.3.2 测试通过率对比	36
5.3.3 变更覆盖率对比	36
5.3.4 测试效率对比	38
5.4 消融实验结果分析	38
5.4.1 各模块贡献度分析	38
5.4.2 自修复能力消融实验	40
5.4.3 LLM 能力消融实验	41
5.5 实验结果讨论	41
5.5.1 方法优势分析	41
5.5.2 局限性分析	42
5.6 本章小结	42
第 6 章 总结与展望	43
6.1 全文总结	43
6.2 主要贡献	43
6.3 研究局限	44
6.4 未来工作展望	44
结 论	46
参考文献	47
附 录	50
附录 A L ^A T _E X 环境的安装	50
附录 B BITHesis 使用说明	50
致 谢	51

第 1 章 绪论

1.1 研究背景与意义

随着软件产业的快速发展，软件系统的规模与复杂度持续增长，软件质量保障面临前所未有的挑战。软件测试作为保证软件质量的关键环节，在软件开发生命周期中占据重要地位。然而，传统软件测试过程高度依赖人工经验，存在效率低、覆盖不足、成本高等问题。据统计，软件测试环节平均消耗整个开发周期 40%-50% 的时间与资源，在安全关键系统中甚至高达 60%-80%^[1]。如何提高测试效率、降低测试成本、保证测试质量，成为软件工程领域亟待解决的重要问题。

近年来，敏捷开发与持续集成（Continuous Integration, CI）模式在软件开发中得到广泛应用，代码变更更加频繁，迭代周期显著缩短。在此背景下，回归测试成为保证软件质量的重要手段。传统的回归测试主要依赖人工编写测试用例，不仅耗时耗力，而且难以快速响应频繁的代码变更。研究表明，开发人员平均需要花费 20%-30% 的工作时间编写和维护测试用例。此外，人工测试用例的质量高度依赖测试人员的经验与技能，存在覆盖不全面、测试点遗漏等问题，难以有效保证测试的有效性与充分性。

代码变更影响分析是回归测试的核心环节之一，其目标在于识别代码变更可能影响的系统范围，从而指导测试资源的分配与测试用例的生成。传统的代码变更分析方法主要基于文本差异比较，仅能识别行级变化，难以反映代码的真实语义与影响范围。例如，当函数内部逻辑发生重构或接口参数发生变化时，仅凭文本差异难以准确识别其对其他模块的影响。这种语义层面的信息缺失，导致测试覆盖不全面或过度测试，影响测试效率与质量。

近年来，人工智能技术，特别是大语言模型（Large Language Model, LLM）与多智能体系统（Multi-Agent System, MAS）的快速发展，为软件测试自动化带来了新的机遇。大语言模型在代码理解、自然语言处理、知识推理等方面展现出强大的能力，为测试用例的自动生成提供了新的技术路径。多智能体系统通过多个 Agent 的协同决策与任务分工，能够处理复杂、多步骤的任务，提高系统的智能化水平与可扩展性^[2]。将大语言模型与多智能体系统相结合，有望实现从代码变更分析到测试用例生成的端到端自动化，显著提升软件测试的效率与质量。

基于上述背景，本课题围绕基于多智能体的软件测试自动化与智能决策方法展开研究，旨在解决传统软件测试过程中对人工经验依赖强、测试覆盖不足及效率较低等问题。通过引入代码语义分析技术与多 Agent 协同机制，实现从代码变更到测试用例生成的自动化与智能化。本研究的意义主要体现在以下几个方面：

(1) 理论意义。本研究将代码语义分析、影响分析、缺陷模式挖掘与大语言模型、多智能体系统相结合，提出一种新的软件测试自动化框架，丰富了软件测试自动化与智能化研究的理论体系。

(2) 技术意义。本研究提出的基于多智能体的测试生成方法，能够有效提升测试用例生成的自动化程度与智能化水平，降低对人工经验的依赖，提高测试效率与质量，具有重要的技术价值。

(3) 应用价值。本研究成果可应用于持续集成环境下的回归测试场景，支持快速迭代开发模式，帮助开发团队及时发现与修复缺陷，降低软件维护成本，具有广阔的应用前景。

1.2 国内外研究现状

1.2.1 软件测试自动化研究现状

软件测试自动化是软件工程领域的重要研究方向，其目标是通过自动化工具与技术，减少或替代人工测试工作，提高测试效率与质量。根据自动化程度的不同，软件测试自动化可分为测试脚本自动化、测试用例生成自动化与测试执行自动化等层次。

在测试用例自动生成方面，传统方法主要分为基于搜索的方法（Search-Based Software Testing, SBST）与基于模型的方法（Model-Based Testing, MBT）^[3]。基于搜索的方法通过优化算法（如遗传算法、模拟退火算法等）在测试用例空间中搜索，以最大化覆盖率或最小化测试数量为目标，自动生成测试用例。代表性工具包括 Pynguin、EvoSuite 等。Pynguin 是针对 Python 语言的单元测试生成工具，采用遗传算法优化测试用例生成过程，支持多种覆盖准则^[4]。EvoSuite 是面向 Java 语言的测试生成工具，采用遗传算法与分支覆盖准则，生成满足特定覆盖目标的测试用例。然而，基于搜索的方法主要关注覆盖率指标，生成的测试用例往往缺乏语义合理性，可读性与可维护性较差。

基于模型的方法通过构建软件系统的形式化模型（如状态机、Petri 网等），基于

模型自动生成测试用例。此类方法需要人工构建模型，工作量大且难以适应频繁的代码变更，在实际应用中存在一定局限性。

近年来，随着深度学习与自然语言处理技术的发展，基于神经网络的测试用例生成方法逐渐受到关注。此类方法通过训练神经网络模型，学习代码与测试用例之间的映射关系，从而自动生成测试用例。代表性工作包括 A3Test、ATLAS 等^[5]。然而，此类方法需要大量标注数据进行训练，且生成的测试用例质量依赖于训练数据的质量，泛化能力有待提升。

大语言模型的出现为测试用例自动生成带来了新的可能。GPT-4、Claude 等大语言模型在代码理解与生成方面展现出强大的能力，能够根据代码或需求描述生成测试用例。研究表明，基于大语言模型的测试生成方法在测试质量与可读性方面优于传统方法，但仍存在生成结果不稳定、缺乏约束与验证等问题^[6]。

1.2.2 代码变更影响分析研究

代码变更影响分析是软件维护与回归测试的重要环节，其目标在于识别代码变更可能影响的系统范围，从而指导测试资源的分配与回归测试的选择^[7]。根据分析方法的不同，代码变更影响分析可分为静态分析与动态分析两大类。

静态分析方法通过分析代码的结构信息（如控制流、数据流、调用关系等），推断代码变更的影响范围。代表性工作包括 Horwitz 等提出的基于系统依赖图（System Dependency Graph, SDG）的影响分析方法^[8]和 Ren 等提出的 Chianti^[9]。Horwitz 等的方法通过构建系统依赖图，利用控制依赖与数据依赖关系分析代码变更对其他模块的影响传播路径。Chianti 通过将代码变更分解为原子变更集合，并结合调用图分析识别受影响的测试用例。静态分析方法的优点在于无需执行程序，分析成本低，但存在误报率较高的问题，难以准确识别动态绑定、反射调用等场景下的影响。

动态分析方法通过执行程序并收集运行时信息（如实际调用路径、数据依赖等），识别代码变更的影响范围。代表性工作包括 Daikon^[10]和 Jalangi^[11]。Daikon 通过动态检测程序不变量，推断代码变更可能违反的约束条件。Jalangi 基于动态分析技术，收集 JavaScript 程序的执行轨迹，支持变更影响分析与回归测试选择。动态分析方法的优点在于分析结果较为准确，但需要执行测试用例，分析成本较高，且依赖于测试用例的覆盖率。

近年来，研究者开始将静态分析与动态分析相结合，构建混合式影响分析方法。

代表性工作包括 Chianti 与 Ekstazi 等^[9,12]。Chianti 通过静态分解识别变更影响范围，再通过动态调用图验证影响是否真实发生。Ekstazi 在静态依赖分析的基础上，结合代码覆盖率信息进行回归测试选择，有效降低了测试执行成本。然而，现有影响分析方法主要关注代码结构层面的影响，对代码语义层面的影响识别能力有限。例如，当函数内部逻辑发生重构时，结构层面的变更可能较小，但对系统行为的影响可能较大。如何将语义信息融入影响分析过程，提高影响分析的准确性，是当前研究的难点之一。

1.2.3 多智能体与大模型在软件工程中的应用

多智能体系统是由多个智能体组成的分布式系统，各智能体通过协同决策与任务分工，共同完成复杂任务。多智能体系统具有自治性、协作性、适应性等特点，在复杂系统建模、任务规划、决策支持等领域得到广泛应用。

近年来，多智能体系统在软件工程中的应用逐渐受到关注。代表性工作包括 AutoGen、MetaGPT 等^[13-14]。AutoGen 是微软提出的通用多智能体框架，支持多个大语言模型 Agent 之间的对话与协作，可应用于代码生成、测试、调试等场景。MetaGPT 是面向软件开发的多智能体系统，通过模拟软件开发团队的角色分工（如产品经理、架构师、工程师等），实现需求分析、设计、编码、测试等任务的自动化。此类工作展示了多智能体系统在软件工程领域的应用潜力，但主要集中在代码生成与需求分析环节，在测试用例生成方面的研究较少。

大语言模型在软件工程中的应用近年来取得显著进展。在测试生成方面，研究者开始探索基于大语言模型的测试用例生成方法。代表性工作包括 CoditT5、ChatTester 等^[15]。CoditT5 基于 CodeT5 模型，根据代码变更自动生成测试用例。ChatTester 基于 ChatGPT，通过多轮对话方式引导测试用例的生成与优化。

然而，现有研究主要将大语言模型作为单一组件使用，缺乏多组件之间的协同机制。在实际的测试生成过程中，需要经过代码变更分析、影响分析、测试规划、测试生成等多个环节，单一模型难以有效处理所有环节。如何构建多智能体协同框架，实现各环节之间的信息流转与任务联动，是当前研究的空白。

综上所述，现有研究在测试自动化、影响分析、大语言模型应用等方面取得了一定进展，但仍存在以下不足：（1）传统测试自动化方法主要关注覆盖率指标，缺乏对测试语义合理性的考虑；（2）现有影响分析方法主要关注结构层面，对语义层面

的影响识别能力有限；（3）大语言模型在测试生成中的应用主要采用单一模型，缺乏多组件协同机制。针对上述问题，本课题提出基于多智能体的软件测试自动化方法，通过代码语义分析、影响分析、缺陷模式挖掘与大语言模型、多智能体系统的深度融合，实现从代码变更到测试用例生成的端到端自动化。

1.3 本文主要研究内容

本课题围绕基于多智能体的软件测试自动化与智能决策方法展开研究，主要研究内容包括以下五个方面：

（1）代码变更分析方法研究。传统基于文本差异的代码分析方法仅能识别行级变化，难以反映代码的真实语义。本研究引入抽象语法树（AST）与 Tree-sitter 技术，对源代码进行语法级解析，将非结构化代码转换为结构化表示^[16]。在此基础上，对变更内容进行语义级建模，识别函数、类、接口及变量等关键程序实体的变化，实现从“文本变更”向“语义变更”的转化，为后续分析提供可靠基础。

（2）影响分析模型设计。为准确评估代码变更的影响范围，构建基于调用关系与依赖关系的影响分析模型。调用关系（Call Graph）用于描述函数或方法之间的调用路径，当底层函数发生变更时，可沿调用链向上追溯受影响的上层模块；依赖关系分析涵盖模块依赖、类依赖及数据依赖等，用于识别因接口或数据变化引发的连锁影响^[8,17]。通过结合调用关系传播路径与依赖关系分析，构建完整的变更影响传播链路，实现从局部变更到全局影响范围的推导。

（3）缺陷模式挖掘与融合机制。为提升测试的针对性与有效性，引入历史缺陷数据，对典型缺陷进行模式化建模（Bug Pattern）。通过分析历史缺陷记录，总结常见缺陷特征，构建缺陷模式库。在实际应用中，将代码变更与缺陷模式进行匹配，从而识别潜在风险点，并指导测试重点的确定。

（4）测试规划与用例生成方法。在获取代码变更信息、影响范围及缺陷模式后，设计测试规划 Agent 进行统一决策。该模块通过融合多源信息生成测试策略，并进一步细化为具体测试点。在此基础上，结合大语言模型与 Prompt 工程方法，将测试点转化为结构化测试用例，包括测试步骤、输入数据及预期结果等^[18]。

（5）多智能体协同框架设计。为提升系统整体智能化水平，构建多智能体协同框架。各 Agent 分别负责代码变更分析、影响分析、测试规划等功能，并通过统一状态管理机制进行调度。在该框架下，各 Agent 共享上下文信息并协同决策，实现信息

闭环与任务联动，从而提高系统整体的分析能力与扩展性。

1.4 本文组织架构

本文共分为六章，各章节内容安排如下：

第一章为绪论。介绍研究背景与意义，分析国内外研究现状，阐述本文的主要研究内容与组织架构。

第二章为相关技术基础。介绍本研究涉及的关键技术，包括代码分析技术（AST、Tree-sitter）、影响分析方法（调用图、依赖分析）、大语言模型与 Prompt 工程、多智能体系统框架（LangGraph）等，为后续章节提供技术支撑。

第三章为系统总体设计。阐述系统的总体架构与设计原则，介绍系统的核心模块及其相互关系，描述数据流与控制流的设计，说明关键技术选型与实现方案。

第四章为关键技术与实现。详细阐述各核心模块的设计与实现，包括代码变更分析模块、影响分析模块、缺陷模式匹配模块、测试用例生成模块、多智能体协同框架等，并给出关键技术细节与实现方案。

第五章为实验与结果分析。介绍实验设计，包括实验对象、评估指标、对比方法等，展示实验结果并进行深入分析，验证所提方法的有效性，讨论实验结果的意义与局限性。

第六章为总结与展望。总结本文的研究工作与主要贡献，分析当前研究的不足之处，展望未来研究方向与改进空间。

各章节之间的逻辑关系如下：第一章阐述研究背景与目标，第二章提供技术基础，第三章给出系统总体设计方案，第四章详细实现各核心模块，第五章通过实验验证方法有效性，第六章总结全文并展望未来工作。

第 2 章 相关技术与方法

2.1 软件测试相关技术

2.1.1 单元测试与集成测试

单元测试（Unit Testing）是软件测试体系中最基础的测试级别，其核心目标是对软件中的最小可测试单元进行验证^[1]。在面向对象编程范式下，单元通常指一个函数、一个类或一组紧密相关的函数集合。单元测试具有以下典型特征：每个测试用例应具有原子性，专注于验证单一功能点；测试用例之间应相互独立，避免共享状态导致测试结果的相互影响；单元测试应在隔离环境中执行，通过模拟（Mock）或桩（Stub）技术排除对外部依赖的依赖。单元测试的优势在于能够快速定位缺陷，降低调试成本，同时也是代码重构的重要保障。

集成测试（Integration Testing）旨在验证多个软件模块或组件之间交互的正确性^[1]。与单元测试不同，集成测试关注的是模块接口、调用协议、数据交换及集成后的系统行为。集成测试能够发现模块接口不匹配、数据格式不一致、调用时序错误等问题，是单元测试与系统测试之间的重要桥梁。在持续集成（Continuous Integration, CI）环境中，集成测试通常配置为自动化执行，及时发现代码集成过程中引入的缺陷。

在回归测试场景中，单元测试与集成测试具有互补作用。单元测试用于验证代码变更对单个函数或类的影响，侧重于局部正确性验证；集成测试则用于验证代码变更对模块间协作的影响，侧重于全局行为验证^[19]。本研究聚焦于面向代码变更的回归测试用例生成，需要同时考虑单元测试级别与集成测试级别的用例生成。

2.1.2 自动化测试框架（Pytest）

Pytest 是 Python 生态中应用最为广泛的自动化测试框架之一，以其简洁的 API 设计、强大的插件生态及丰富的功能特性著称^[20]。相较于 Python 标准库中的 unittest 框架，Pytest 采用更加灵活的设计理念，支持基于函数的测试用例定义，降低了测试代码的编写门槛。Pytest 的核心设计原则包括：约定优于配置、失败即停、以及详细的断言重写。

Pytest 的 Fixture 机制是其最具特色的功能之一，Fixture 用于提供测试所需的上

下文环境或依赖资源，支持多种作用域（function、class、module、session）及参数化配置^[20]。此外，社区提供 `pytest-asyncio`、`pytest-mock`、`pytest-xdist` 等大量插件。本研究基于 `Pytest` 构建测试执行环境。

2.2 代码变更分析技术

2.2.1 Unified Diff 解析

代码变更分析的第一步是对版本控制系统产生的差异文件进行解析。`Git` 作为目前最流行的分布式版本控制系统，其差异输出格式（`Diff Format`）是代码变更分析领域的事实标准^[21]。`Git Diff` 采用 `Unified Diff` 格式进行展示，新增行以前缀 `+` 标识，删除行以前缀 `-` 标识，上下文行以空格前缀标识。

`Python` 标准库中的 `difflib` 模块提供了差异计算的基础功能^[22]。为实现符合 `Unified Diff` 规范的解析与处理，社区广泛使用 `Python` 包 `unidiff`，该库提供了从统一差异格式到结构化数据模型的双向转换能力^[23]。通过 `unidiff`，开发者可将原始 `Diff` 文本解析为包含 `PatchedFile`、`SourceFile`、`Hunk` 等对象的数据结构，每个对象封装了文件路径、变更块起止位置、具体变更行等属性信息。

`Unified Diff` 解析在实际应用中面临若干技术挑战：变更块的重叠与合并问题、二进制文件与文件重命名的处理、编码与换行符问题等。本研究在代码变更解析模块中集成了 `Unified Diff` 解析功能，用于将 `Git` 提交产生的差异文件转换为结构化的变更表示。

2.2.2 代码变更图与依赖关系建模

代码变更的影响分析需要借助程序结构模型来追踪变更的传播路径。程序依赖图（`Program Dependence Graph`, `PDG`）是这一领域最具代表性的表示模型^[24]。`PDG` 将程序的控制流依赖与数据流依赖统一建模，通过 `PDG` 可以精确追踪程序中任意语句对其他语句的影响范围。

调用图（`Call Graph`）是描述函数或方法调用关系的重要模型^[25]。在调用图中，节点表示函数实体，有向边表示调用关系。调用图可分为静态调用图和动态调用图两种类型。静态调用图通过解析代码中的函数调用表达式构建，覆盖所有可能的调用路径；动态调用图则记录程序实际执行过程中的调用序列，覆盖范围有限但准确性更高。在变更影响分析中，调用图支持自底向上的影响传播。

除调用关系外，依赖关系还包括模块依赖、类依赖及数据依赖等多种类型^[26]。本研究构建了综合依赖模型，整合调用图、控制流依赖与数据流依赖等多种关系表示，通过图遍历算法实现变更影响范围的有效识别。

影响分析方法可分为基于覆盖率的测试选择与基于传播的变更影响分析两类^[7]。基于覆盖率的方法通过比较变更前后的代码覆盖率差异来确定受影响的测试用例集^[27]。基于传播的方法则从变更点出发，沿程序依赖关系图进行传播，识别所有直接或间接受影响的程序实体^[28]。本研究综合采用两类方法。

2.2.3 代码语义表示方法（AST 与 LLM）

代码的结构化表示是进行语义级代码分析的基础。抽象语法树（Abstract Syntax Tree, AST）将源代码转换为树形结构，其中每个节点对应源代码中的一个语法构造^[29]。AST 在保留代码语法结构的同时移除了无关的格式信息，使得程序分析可以在结构化层面进行。Python 的 AST 模块可将源代码解析为 `ast.Module` 根节点下的层次结构^[30]。

Tree-sitter 是一个用于构建多语言解析器的增量式解析系统^[31]。与传统的解析器不同，Tree-sitter 具有语言无关的解析框架、增量解析能力、错误容忍机制等特点，使其特别适用于代码变更分析场景。

大语言模型（Large Language Models, LLMs）的出现为代码语义表示提供了新的技术路径^[32]。预训练代码模型（如 Codex、CodeT5、StarCoder 等）在大规模代码语料上进行预训练，学习到了代码的语义表示与生成能力^[33]。代码嵌入（Code Embedding）将代码片段映射为稠密向量表示，使得代码之间的语义相似性可通过向量距离进行度量^[34]。

然而，纯文本表示的 LLM 在处理结构化程序代码时存在固有限制。常见思路包括基于 AST 的序列化、基于图神经网络的程序图编码以及多模态信息融合等。本研究结合 Tree-sitter 的结构化解析与 LLM 的语义理解，实现从语法节点到语义推理的衔接。

2.3 多智能体系统技术

2.3.1 基于 LangGraph 的工作流建模

LangGraph 是 LangChain 生态中用于构建有状态、多角色交互应用的工作流编排框架^[35]。与传统的线性链条式（Chain）处理流程不同，LangGraph 采用图结构对复杂工作流进行建模，其中节点（Node）表示处理单元，边（Edge）表示控制流与数据流。这种图结构天然支持条件分支、循环迭代及多路径并行等复杂控制模式。

LangGraph 的核心抽象包括 StateGraph、节点（Node）和边（Edge）三个层次：状态模式常借助 Pydantic 等机制描述全局状态^[36]；节点以函数形式实现状态的读入与更新；边刻画普通迁移、条件分支及入口/终止等控制关系。框架支持惰性求值与状态持久化。

本研究采用 LangGraph 作为多智能体协同框架的核心基础设施。通过 StateGraph 定义测试生成工作流的全局状态结构，工作流中的各个处理阶段均建模为独立节点，通过边定义节点间的数据流向与控制依赖。这种基于图的工作流建模方式既保证了处理流程的清晰性，又为后续的功能扩展与优化提供了灵活性。

2.3.2 多 Agent 协同机制

多智能体系统(Multi-Agent System, MAS)是由多个具有自主能力的智能体(Agent)组成的计算系统，智能体之间通过协作完成复杂任务^[37]。与单一智能体相比，多智能体系统在问题分解、信息集成、容错冗余及并行处理等方面具有显著优势^[38]。

智能体的角色定义是多智能体系统设计的基础^[39]。每个智能体通常被赋予特定的角色，承担特定的功能职责。在软件工程应用场景中，常见的角色包括分析型 Agent（负责理解需求、分析问题）、规划型 Agent（负责制定执行计划、分解任务）、执行型 Agent（负责具体操作的实施）、验证型 Agent（负责检查结果正确性）及协调型 Agent（负责管理多个 Agent 的协作流程）^[13]。

信息共享是多智能体协同的核心机制^[40]，常见模式包括点对点消息、黑板、发布-订阅与消息队列等。在基于 LLM 的系统中，常通过共享上下文或对话历史传递信息^[14]。对于可并行的子任务可并行执行，存在依赖的子任务则按拓扑顺序串行^[41]。当多个 Agent 结论不一致时，可采用权威裁定、投票、置信度比较或协商等策略^[42]。

本研究构建的多智能体测试生成系统包含 11 个专业 Agent，分别负责代码变更

解析、AST 分析、影响范围推理、测试策略规划、测试用例生成、测试执行、结果评估、缺陷模式匹配、代码检索、上下文管理和报告生成等功能。

2.4 大语言模型技术

2.4.1 代码理解与结构化生成

大语言模型在代码理解任务中的能力源于其在大规模代码语料上的预训练^[32]。预训练阶段，模型学习到编程语言的语法规则、语义模式及常见的编码范式^[43]。代码理解任务包括代码摘要、代码搜索、代码补全及代码翻译等^[18]。

代码嵌入将代码映射为稠密向量，使语义相近片段在向量空间中彼此接近^[44]。

结构化生成要求模型输出符合既定代码形态与项目规范。常见对策包括约束解码、提示工程与生成后校验等^[45-46]。本研究结合提示模板与生成后检查，以约束测试代码形态。

检索增强生成（RAG）通过从外部库检索相关片段并拼入提示，扩展模型可用知识^[47]。本研究在流水线中集成了这一机制。

2.4.2 测试用例生成与修复

基于大语言模型的测试用例生成已成为软件测试领域的研究热点^[48]。其核心难点包括：如何把变更信息组织为有效提示、如何约束输出符合工程规范、如何评价充分性等。

提示设计需要兼顾任务说明、上下文、格式约束与示例等要素^[49]；也可采用思维链/分步规划等提示策略以提升筹划性。

测试用例修复用于应对生成中的语法与语义问题^[50]，既可通过静态分析修补，也可结合执行反馈迭代。Self-healing 等研究总结了失败模式并给出自动修复策略^[51]。

充分性常用覆盖率等指标刻画；亦可通过故障注入考察检出能力。本研究以覆盖率与执行成功率为主，并在失败时进入修复回路。

综上所述，本章介绍了与本研究相关的核心技术与方法。软件测试技术为研究提供了测试理论与工程实践基础；代码变更分析技术为影响范围推理提供了技术支持；多智能体系统技术为复杂测试生成任务的协同处理提供了架构框架；大语言模型技术为代码理解与测试用例生成提供了智能能力。这些技术与方法相互结合，共同构成本研究的技术基础。

第3章 基于代码变更的影响分析方法

3.1 问题定义与总体架构

3.1.1 问题定义

代码变更影响分析的核心问题可形式化定义为：给定一个代码变更集合 Δ ，准确识别所有可能受该变更影响的目标程序实体集合 T ，并量化每个目标实体的受影响程度。本研究中，程序实体 T 涵盖函数（Function）、类（Class）、模块（Module）及接口（Interface）等多个粒度层次。

形式化表述如下：设程序 P 由实体集合 $E = \{e_1, e_2, \dots, e_n\}$ 组成，代码变更 Δ 包含一系列文件级别的修改 $\delta_1, \delta_2, \dots, \delta_m$ 。影响分析的输出为目标实体集合 $T \subseteq E$ ，以及每个目标实体 $t \in T$ 的影响评分 $I(t) \in [0, 1]$ ，表示该实体受到变更影响的可能性与程度。

代码变更根据其意图可分为以下三类：

- **缺陷修复型（BUG_FIX）**：旨在修复已有缺陷的代码变更，通常涉及错误处理逻辑、边界条件检查等。
- **功能新增型（FEATURE）**：旨在引入新功能的代码变更，通常涉及新增接口、扩展逻辑等。
- **重构优化型（REFACTOR）**：旨在改善代码结构或性能的代码变更，功能行为应保持不变。

传统方法主要依赖文本级 Diff 比较，仅能识别行级代码变化，难以捕获代码的真实语义变更。端到端黑箱式的影响分析方法虽然简化了处理流程，但存在可追溯性不足的问题。

3.1.2 核心理念：结构化中间表示

本研究提出基于结构化中间表示的代码变更影响分析方法，将影响分析过程分解为多个明确界定的阶段。这种方法论的优势体现在三个方面：

可追溯性（Traceability）：结构化中间表示使得分析人员能够追踪任意影响路径的完整推导过程。

可审计性 (Auditability): 每个中间表示都附带元数据（如置信度评分、来源说明、时间戳等），支持对分析过程进行事后审计。

可组合性 (Composability): 结构化中间表示支持模块化组合，不同的分析阶段可以独立开发、测试与优化。

本研究提出的结构化中间表示框架包含以下核心层次：

- **结构化变更表示 (SCR)**: 将 Unified Diff 解析为文件级、块级、行级的结构化模型。
- **变更图表示 (CGR)**: 基于代码依赖关系构建变更影响的有向图结构。
- **语义单元表示 (SUR)**: 将变更代码与受影响代码映射为原子语义单元。
- **结构化影响报告 (SIR)**: 最终输出的结构化影响分析结果。

这四层表示形成一条完整的链条： $SCR \rightarrow CGR \rightarrow SUR \rightarrow SIR$ ，每层都有明确的语义定义与转换接口。

3.1.3 总体架构设计

图 3-1 展示了代码变更影响分析的总体架构。该架构遵循流水线式的处理流程，各模块依次处理前一模块的输出，并产生下一模块所需的输入。

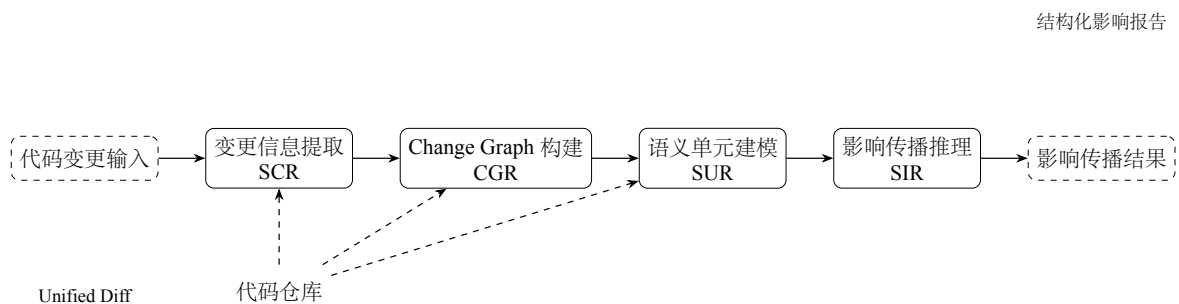


图 3-1 基于结构化中间表示的影响分析总体架构

系统接受两类输入：**Git** 提交产生的 **Unified Diff** 格式代码变更，以及被测代码仓库的当前状态（用于一致性校验）。输入数据首先经过代码变更信息提取模块的处

理，生成结构化变更表示（SCR）。SCR 包含变更文件的列表、变更代码块的位置信息，以及通过 AST 解析提取的程序实体（函数签名、类定义、变量声明等）。

变更信息提取模块的输出传递至 Change Graph 构建模块，构建全局的程序结构图并生成变更图表示（CGR）。语义单元建模模块对变更代码进行语义级分析，生成语义单元表示（SUR）。影响传播模块计算每个受影响实体的风险评分与优先级。最终，分析结果生成模块整合前序各层的中间表示，输出结构化的影响报告（SIR）。

3.2 代码变更信息提取

代码变更信息提取是影响分析的起点，其目标是将非结构化的 Unified Diff 文本转换为结构化的变更表示。

3.2.1 Unified Diff 解析与结构化

Git 生成的 Unified Diff 格式包含文件头（Hunk Header）和变更块（Hunk Body）两部分。本研究采用 Python 的 `unidiff` 库进行解析^[23]。解析过程的核心数据结构定义如下：

```
1 class ParsedDiff:
2     source_file: str      # 变更前文件路径
3     target_file: str      # 变更后文件路径
4     hunks: List[Hunk]     # 变更块列表
5
6 class Hunk:
7     source_start: int      # 源文件起始行号
8     source_length: int     # 源文件块长度
9     target_start: int      # 目标文件起始行号
10    target_length: int     # 目标文件块长度
11    source_lines: List[str] # 源文件行内容
12    target_lines: List[str] # 目标文件行内容
13    line_type: str         # 行类型: 'add'/'remove'/'context'
```

解析流程包含以下关键步骤：Diff 文本标准化、变更块提取、行级内容解析、结构化输出。

3.2.2 变更文件与代码块建模

本研究构建了三层变更模型：

文件级变更模型描述了变更文件的整体信息，包括文件路径、变更类型及变更

统计。**块级变更模型**描述了单个变更块的详细信息，包括块在文件中的位置、涉及的函数或类。**行级变更模型**记录了每一行的变更细节，为 AST 节点定位提供精确的行号信息。

三层模型的形式化表示为：

$$\Delta_{file} = \{f_1, f_2, \dots, f_n\}$$

$$\Delta_{hunk}(f_i) = \{h_1, h_2, \dots, h_m\}$$

$$\Delta_{line}(h_j) = \{l_1, l_2, \dots, l_k\}$$

3.2.3 变更实体抽取

为获取变更实体的语义信息，需要对变更代码进行 AST 解析^[29]。变更实体抽取的过程分为两步：上下文恢复与 AST 解析。

上下文范围的确定遵循以下规则：从变更块所在行向上遍历寻找函数或类定义的起始位置；向下遍历寻找函数或类定义结束的标记；对于 Python 代码，利用缩进层级判断函数边界的结束位置。

本研究集成 Tree-sitter 作为多语言通用解析引擎^[31]，具有多语言支持、增量解析、容错能力等特点^[30]。AST 解析后，通过遍历语法树节点提取关键实体信息，包括函数定义、类定义、函数调用、导入声明、变量赋值等。

抽取结果封装为变更实体对象，结构定义如下：

```

1 class ChangeEntity:
2     entity_type: str          # 实体类型：function/class/method/variable
3     entity_name: str         # 实体名称
4     entity_path: str         # 实体所属文件路径
5     location: tuple          # 实体在文件中的位置 (start_line, end_line)
6     change_type: str         # 变更类型：added/modified/deleted
7     diff_lines: List[tuple]  # 变更行信息 (line_no, line_content, change_type)
8     metadata: dict           # 额外元信息（函数签名、参数列表等）

```

3.3 Change Graph 构建

Change Graph 是本研究提出的核心数据结构，用于表示代码实体之间的依赖关系与变更影响传播路径。

3.3.1 代码依赖关系建模

代码依赖关系是影响传播的基础，不同类型的依赖关系具有不同的传播特性：

调用依赖描述函数或方法之间的调用关系，具有传递性，是影响传播的基础。**导入依赖**描述模块之间的导入关系。**继承依赖**描述类与类之间的继承关系。**数据依赖**描述变量或数据结构的共享关系。

形式化地，代码依赖关系可建模为带权有向图 $G = (V, E, W)$ ，其中 $V = \{v_1, v_2, \dots, v_n\}$ 表示程序实体节点集合， $E \subseteq V \times V$ 表示有向边集合， $W : E \rightarrow \mathbb{R}^+$ 表示边权重函数。

边的权重 $W(e)$ 根据依赖类型计算：调用依赖的权重通常设为 1.0；导入依赖根据导入符号的数量调整；继承依赖根据是否为覆盖方法调整。

3.3.2 Change Graph 构建方法

Change Graph 的节点类型包括 `FileNode`（表示源代码文件）、`FunctionNode`（表示函数或方法）、`ClassNode`（表示类定义）、`ModuleNode`（表示模块）。边类型包括 `contains`、`calls`、`inherits`、`imports`、`uses` 等。

构建算法首先遍历仓库中的所有源文件，构建基础的文件-实体节点结构。然后，通过调用链分析构建函数间的调用边，通过导入分析构建模块间的导入边。最后，为所有边计算权重值，完成 Change Graph 的构建。

构建完成的 Change Graph 支持正向追溯（Forward Trace）、反向追溯（Backward Trace）、路径查询（Path Query）、影响子图提取（Impact Subgraph Extraction）等查询操作。

3.4 语义单元建模方法

语义单元是影响分析中的关键概念，代表代码变更或受影响代码的原子语义功能块。

3.4.1 原子语义单元定义

原子语义单元（ASU）是指在语义层面不可再分的代码功能块。ASU 的划分遵循功能单一性、边界清晰性、可测试性等原则。

基于代码结构，ASU 可分为函数语义单元、类语义单元、模块语义单元三种基本类型。

ASU 的形式化定义如下：

$$ASU = (id, type, semantic_content, contracts, tags, risk_score)$$

其中， $type \in \{function, class, module\}$ ， $semantic_content$ 是语义内容描述， $contracts$ 是输入输出契约， $tags$ 是语义标签集合， $risk_score$ 是风险评分。

3.4.2 语义标签体系

本研究设计了多维的语义类型与标签体系。

语义单元类型定义了代码的主要功能类别：

- **API 型（API）**：暴露给外部调用的接口函数。
- **业务逻辑型（Business Logic）**：核心业务规则实现。
- **数据处理型（Data Processing）**：数据的转换、过滤、聚合等操作。
- **异常处理型（Exception Handling）**：异常捕获、抛出与处理逻辑。
- **配置型（Configuration）**：配置项的定义与管理。
- **工具型（Utility）**：通用工具函数。

变更语义标签用于描述代码变更的语义特征：

- **api_signature_changed**：API 签名变更，需验证接口兼容性。
- **dependency_call_changed**：依赖调用变更，需验证 Mock 策略。
- **exception_handling_changed**：异常处理变更，需验证异常场景。
- **logic_branch_changed**：逻辑分支变更，需验证分支覆盖。

- `null_check_added`: 空值检查新增, 需验证防御性编程。

语义单元的提取过程结合静态分析与 LLM 推断两种方法, 以兼顾效率与准确性。语义标签采用多值标注的方式, 一个 ASU 可以同时拥有多个标签。

3.5 影响传播与风险评估模型

影响传播是变更影响分析的核心环节, 目标是在 **Change Graph** 上追踪变更的传播路径, 确定所有受影响的程序实体。

3.5.1 跨符号影响传播机制

本研究设计了多层次的影响传播机制, 覆盖函数级、类级与模块级三个传播维度。

函数级传播从被变更的函数出发, 沿调用链向上追溯所有调用该函数的上层函数。传播规则定义如下:

$$I' = I \cup \{f \mid \exists g \in I : calls(g, f) \wedge f \in C\}$$

其中, $calls(g, f)$ 表示函数 g 调用了函数 f 。

传播过程支持多种调用关系类型的差异化处理: 直接调用 (权重 1.0)、虚函数调用 (权重 0.8)、回调函数 (权重 0.6)。

类级传播在类层次上追踪变更影响, 包括向下传播 (类内) 和向上传播 (类外) 两个方向。

模块级传播在包或模块层次上追踪变更影响:

$$M' = M \cup \{m \mid \exists n \in M : imports(n, m) \wedge changed_api(m)\}$$

三层传播的协同工作机制确保了影响分析在不同粒度上都得到完整覆盖。

3.5.2 影响范围限定策略

本研究设计了多种影响范围限定策略:

传播深度限制: 设定最大传播深度 D_{max} , 默认设置为 5。

权重阈值过滤: 设定权重阈值 $w_{threshold}$, 默认设置为 0.5。

白名单机制: 允许用户指定不受传播影响的实体集合。

黑名单机制: 允许用户指定强制标记为受影响的实体集合。

变更类型过滤：根据变更的类型排除不可能产生影响的传播路径。

3.5.3 风险评分与优先级量化

本研究设计了综合考虑多维因素的风险评分模型：

$$Risk(e) = w_1 \cdot ChangeType(e) + w_2 \cdot PropagationDepth(e) + w_3 \cdot DefectHistory(e) + w_4 \cdot SemanticComplexity(e)$$

其中各因子的含义与计算方法：

- $ChangeType(e)$ ：变更类型评分，删除 1.0，修改 0.4-0.8，新增 0.3。
- $PropagationDepth(e)$ ：传播链中的深度，采用指数衰减模型。
- $DefectHistory(e)$ ：历史缺陷密度，反映实体本身的脆弱性。
- $SemanticComplexity(e)$ ：语义复杂度，根据 ASU 的语义标签计算。

本研究默认设置为 $w_1 = 0.35, w_2 = 0.25, w_3 = 0.20, w_4 = 0.20$ 。

基于风险评分，受影响实体可划分为以下优先级层次：

- **P0（高风险）**： $Risk(e) \geq 0.7$ ，需要优先测试。
- **P1（中风险）**： $0.4 \leq Risk(e) < 0.7$ ，需要测试。
- **P2（低风险）**： $Risk(e) < 0.4$ ，可选择性测试。

3.6 影响分析结果生成与结构化输出

影响分析完成后，需要将分析结果组织为结构化的输出格式，供下游模块使用。

3.6.1 结构化输出设计

本研究设计了 JSON 格式的结构化输出，涵盖影响分析的主要结果与元信息，包括分析元信息、影响摘要、种子实体、语义单元、变更图、传播路径与测试建议等层次。

输出文件保存为 `impact_analysis_result.json`，供后续模块读取使用。

3.6.2 与下游模块的接口规范

本研究定义了清晰的模块间接口规范，确保了模块间的松耦合设计与独立演进能力。

测试规划模块接口返回按优先级排序的受影响实体列表。**测试检索模块接口**返回与查询实体语义相似的历史测试用例列表。**测试生成模块接口**返回指定语义单元的完整信息。

第 4 章 多智能体测试生成系统设计与实现

4.1 系统总体架构设计

本章围绕“面向代码变更的多智能体回归测试用例生成”这一核心任务，阐述原型系统的总体架构设计。

4.1.1 系统总体架构

本系统采用四层架构模型，自顶向下依次为：接入与展示层、编排与状态层、智能体执行层、运行与评估支撑层。该分层设计遵循关注点分离原则，使得各层职责明确、接口清晰，便于独立演进与测试。

接入与展示层基于 FastAPI 框架构建，负责接收外部请求并返回处理结果。该层对外暴露统一的 RESTful API 接口，支持同步调用与流式响应两种模式。请求与响应的数据模型均采用 Pydantic 进行定义与校验。

编排与状态层采用 LangGraph 的 StateGraph 机制实现，负责定义多智能体之间的工作流拓扑与状态传递逻辑。StateGraph 维护一个贯穿整个工作流的全局状态对象 WorkflowState，承载从原始 diff 到最终评估结果的完整数据。

智能体执行层包含 12 个命名节点，按照流水线拓扑顺序依次执行。各智能体均为独立的 LLM 调用单元，负责完成特定的分析、规划或生成任务。智能体之间不直接通信，而是通过读写共享状态实现协作。

运行与评估支撑层提供测试执行、覆盖率采集与结果评估的能力。该层封装了 pytest 的调用接口，支持在工作树隔离环境中执行生成的测试用例，并采集行覆盖率、分支覆盖率等指标。

在技术选型上，大模型调用层采用 OpenAI 兼容客户端设计，支持 DeepSeek、智谱等主流模型的灵活切换；状态管理层基于 Pydantic 模型实现数据结构定义与运行时校验；持久化层采用 SQLite 记录工作流执行轨迹。

与单智能体方案相比，本系统的架构设计具有以下差异特征：采用流水线分工模式，降低了单个智能体的认知负荷；所有中间表示均显式存储于全局状态中，可供后续审计与消融实验使用；关键能力以可配置开关的形式嵌入状态对象。

接入与展示层

FastAPI + Pydantic

编排与状态层

LangGraph StateGraph + WorkflowState

智能体执行层

12 个命名 Agent 节点

运行与评估支撑层

pytest + 覆盖率采集 + SQLite 持久化

图 4.1 系统四层架构示意图
Figure 4.1 Four-layer Architecture of the System

4.1.2 多智能体协同流程

完整的工作流节点序列为：CodeChangeAgent → ProjectProbeAgent → ImpactAnalysisAgent → BugPatternAgent → TestPlanningAgent → TestPrioritizationAgent → RetrievalAgent → TestGenAgent → TestRepairAgent → ExecutionAgent → EvaluationAgent → FeedbackAgent → END。

各智能体的职责划分如下：

- CodeChangeAgent：负责解析原始 diff 并构建变更理解结构
- ProjectProbeAgent：提取项目结构与依赖信息
- ImpactAnalysisAgent：分析代码变更的影响范围，生成影响图
- BugPatternAgent：识别潜在缺陷模式
- TestPlanningAgent：基于影响分析结果生成结构化测试计划
- TestPrioritizationAgent：对测试用例进行优先级排序
- RetrievalAgent：检索相似测试作为生成参考
- TestGenAgent：核心生成单元，负责产出 pytest 测试代码
- TestRepairAgent：对生成的测试进行修复与优化
- ExecutionAgent：在隔离环境中执行测试并采集结果

- **EvaluationAgent**: 汇总评估指标
- **FeedbackAgent**: 生成最终报告并更新全局状态

数据在各节点之间的传递通过 **WorkflowState** 实现。以变更理解阶段为例, **CodeChangeAgent** 读取原始 diff, 输出变更图与变更分析字段写入状态; **ImpactAnalysisAgent** 读取上述字段, 执行影响分析后, 将影响图与语义单元目录写入状态; 后续节点依次读取并丰富相关字段, 形成端到端的数据流闭环。

系统定义了三种核心的中间表示数据结构: 变更图 (**ChangeGraph**) 以代码变更的最小语义单元为节点; 影响图 (**impact_graph**) 按分层组织方式组织影响范围; 语义单元目录 (**SemanticUnit Catalog**) 将代码实体按类型划分为 **config**、**exception**、**api**、**data_processing** 等类别, 每个单元包含优先级评分。

结构化测试用例 (**StructuredTestCase**) 作为影响侧与生成侧之间的桥梁, 其字段设计包含 **symbol_id**、**semantic_unit_ids** 等影响侧标识, 以及 **expected_behavior**、**pre-requisite** 等生成侧约束。

如图 4-1所示, **StructuredTestCase** 的核心字段包括用例标识符、目标符号、测试分层、优先级、结构化输入、Mock 说明、断言列表和语义单元列表等组成部分。

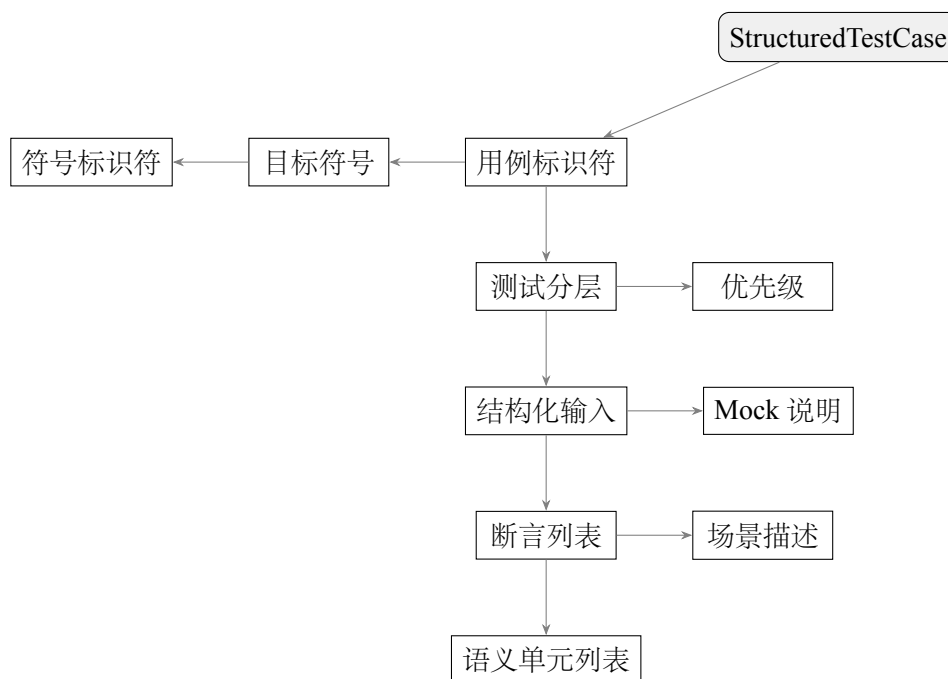


图 4-1 结构化测试用例模型

4.1.3 质量门控机制

为确保生成的回归测试用例具备基本的可用性与有效性，本系统在流水线中嵌入多阶段质量门控机制，覆盖语法校验、静态修复、执行验证与量化评估等环节。

语法正确性检查位于 `TestRepairAgent` 之前。系统使用 `Python` 的 `AST` 模块对生成的代码进行解析，验证其语法合法性。若解析失败，则将失败信息记录于状态对象，并转交至 `TestRepairAgent` 进行处理。

`TestRepairAgent` 专门负责处理静态层面与执行前可侦测的问题。其修复范围包括：语法错误修正、导入语句补全、`fixture` 依赖消解、以及代码风格规范化。

执行结果质量评估由 `EvaluationAgent` 完成，计算以下核心指标：通过率、变更行覆盖率、变更函数覆盖率、测试选择精准度（F1 分数）。

表 4-1 质量门控指标体系

指标类别	指标名称	计算方式	阈值说明
语法质量	AST 解析通过率	成功解析数 / 总生成数	$\geq 95\%$
执行质量	用例通过率	通过用例数 / 总用例数	$\geq 80\%$
覆盖率	变更行覆盖率	覆盖变更行 / 总变更行	$\geq 70\%$
覆盖率	变更函数覆盖率	覆盖函数数 / 总函数数	$\geq 80\%$
精准度	Selection F1	$2 \times P \times R / (P + R)$	≥ 0.75

为支持消融实验与方案对比，系统定义了多个可配置开关字段，包括 `retrieval_enabled`、`bug_pattern_enabled`、`test_repair_enabled` 等。

4.2 核心 Agent 模块设计

本系统采用流水线架构，将回归测试生成任务分解为七个阶段。每个阶段由专门的 Agent 负责，通过读写共享的 `WorkflowState` 进行状态传递与协作。

4.2.1 变更理解阶段

变更理解阶段是整个流水线的入口，负责解析代码变更并构建结构化的变更表示。

4.2.1.1 CodeChangeAgent（变更理解）

CodeChangeAgent 作为流水线的首个 Agent，承担着解析代码变更并建立变更语义模型的任务。其核心入口函数为 `ingest_change`。

该 Agent 的核心处理流程分为四个主要步骤：

1. `parse_unified_diff` 函数负责解析统一的 diff 格式，提取变更文件列表和变更块
2. `check_diff_worktree_consistency` 函数验证解析得到的 diff 与工作树状态的一致性
3. `_build_change_analysis` 方法构建变更分析结果
4. `build_change_graph` 方法将分析结果转化为结构化的变更图

该 Agent 的输入状态字段为 `repo_path` 和 `diff`，输出状态字段包括 `changed_files`、`diff_hunks`、`change_analysis` 和 `change_graph`。

CodeChangeAgent 的关键设计在于变更图的数据结构设计。变更图的节点类型包含文件节点、符号节点、种子节点和测试关注点节点四类，边上携带权重与关系类型标签。

4.2.1.2 ProjectProbeAgent（项目探测）

ProjectProbeAgent 负责探测代码仓库的技术栈信息，为后续测试生成提供必要的环境配置依据。其核心流程以 `_build_profile` 方法为主导，探测并推断项目的编程语言、框架类型、模块根路径、依赖声明行，形成完整的项目画像。

该 Agent 读取 CodeChangeAgent 输出的 `changed_files` 字段，用于依赖推断和符号采集。写出的状态包括 `project_profile` 和预配置虚拟环境信息。

4.2.1.3 ImpactAnalysisAgent（影响分析）

ImpactAnalysisAgent 负责分析代码变更的影响范围，识别需要重点测试的语义单元。其处理流程首先调用 `build_impact_graph` 方法构建影响图。最终输出影响图和语义单元目录。

语义单元目录的类型分类体系包括：`config`（配置项与参数变更）、`exception`（异常处理逻辑变更）、`api`（接口签名与行为变更）、`data_processing`（数据处理与转换逻辑）。

辑变更)、`business_logic` (核心业务规则变更)、`integration` (模块间集成点变更)。

该 Agent 的关键设计体现在语义单元的 `priority_score` 由风险系数、变更权重、中心性指标、引用频次和集成风险五个维度综合计算得出。

4.2.2 测试规划阶段

测试规划阶段基于变更理解的结果，制定测试策略与优先级。

4.2.2.1 BugPatternAgent (缺陷模式识别)

`BugPatternAgent` 负责识别代码变更可能引入的典型缺陷模式，为测试用例设计提供针对性指导。该 Agent 利用语义单元目录和风险排序列表构建查询签名，采用向量相似度检索方法寻找匹配的缺陷模式案例。

内置的规则模板覆盖了空指针解引用、数组边界条件、异常处理不完善、并发竞态条件、资源泄漏等常见缺陷类型。当规则模板匹配不足时，可选的 LLM 精炼步骤能够根据具体代码上下文生成更具针对性的缺陷模式假设。

4.2.2.2 TestPlanningAgent (测试规划)

`TestPlanningAgent` 负责将影响分析结果转化为结构化的测试计划，是测试生成的核心依据。

该 Agent 的处理流程设计为幂等操作：若测试计划已存在则直接跳过。具体流程中，Agent 首先基于影响测试计划和语义单元目录生成结构化的测试用例列表。每个 `StructuredTestCase` 包含场景描述、测试目标、所需语义单元标识等信息。

算法 1 `TestPlanningAgent` 测试规划生成

```
1: Procedure plan_tests(state)
2: if state.test_plan  $\neq \emptyset$  then
3:   return state
4: end if
5: structured_cases  $\leftarrow []$ 
6: for all impact_item  $\in$  state.impact_test_plan do
7:   unit_ids  $\leftarrow$  resolve_semantic_unit_ids(impact_item)
8:   case  $\leftarrow$  StructuredTestCase(scenario, unit_ids)
9:   structured_cases  $\leftarrow$  structured_cases  $\cup$  {case}
10: end for
11: state.test_plan  $\leftarrow$  flatten_test_plan(state.structured_test_plan)
12: return state
```

4.2.2.3 TestPrioritizationAgent（测试优先级）

TestPrioritizationAgent 负责对生成的测试计划进行优先级排序，指导测试资源的高效分配。

该 Agent 的核心评分逻辑综合考虑多个维度：为每个测试用例关联的符号及其对应的语义单元计算优先级分数，该分数由语义单元的 `priority_score` 乘以符号在变更图中的中心性指标得出。

优先级分档策略（P0/P1/P2 三档）是该 Agent 的关键设计。P0 档对应核心业务逻辑和关键 API 变更；P1 档对应一般性功能变更；P2 档对应低风险的配置变更。

4.2.3 测试生成阶段

测试生成阶段是流水线的核心环节，负责基于测试计划和变更上下文生成可执行的测试代码。

4.2.3.1 RetrievalAgent（上下文检索）

RetrievalAgent 负责从代码仓库中检索与变更相关的上下文信息，为测试生成提供项目背景知识。

该 Agent 的处理流程首先从变更路径、测试计划等来源抽取关键词序列。Agent 扫描代码仓库收集候选的 Python 源文件和测试文件集合，采用基于词命中的评分机制计算每个片段与查询关键词的相关度，选取 Top-N 得分最高的片段组成检索结果。

4.2.3.2 TestGenAgent（测试生成）

TestGenAgent 是流水线的核心生成 Agent，负责基于结构化测试计划和变更上下文生成可执行的测试代码。

当大语言模型可用时，Agent 调用 `_llm_generate` 方法构造包含项目画像、结构化测试计划、diff 上下文和检索片段的 Prompt，发送给 LLM 生成测试代码。生成的测试代码随后经过一系列清洗处理，包括去除多余解释文本、修正 `import` 错误、规范函数命名等。

Prompt 工程是该 Agent 的核心技术要点。高质量的 Prompt 需要综合项目画像提供领域背景、结构化测试计划明确测试意图、diff 上下文呈现具体变更、检索片段补充项目规范。

算法 2 TestGenAgent 测试生成流程

```
1: Procedure generate_tests(state)
2: prompt  $\leftarrow$  _llm_generate(project_profile, structured_test_plan, change_analysis, diff_hunks, retrieved_context)

3: raw_tests  $\leftarrow$  call_llm(prompt)
4: tests  $\leftarrow$  []
5: for all raw  $\in$  raw_tests do
6:   clean  $\leftarrow$  _sanitize_comments(raw)
7:   clean  $\leftarrow$  _sanitize_imports(clean)
8:   if syntax_check(clean) then
9:     tests  $\leftarrow$  tests  $\cup$  {GeneratedTestFile(clean)}
10:  end if
11: end for
12: state.generated_tests  $\leftarrow$  tests
13: return state
```

4.2.4 测试执行阶段

测试执行阶段负责运行生成的测试用例并收集执行结果。

4.2.4.1 ExecutionAgent（测试执行）

ExecutionAgent 负责在正确配置的虚拟环境中执行测试用例，并收集结构化的执行结果。

该 Agent 的核心处理流程首先确定 Python 解释器路径。Agent 在 .mutiagent/reports/ 目录下创建以时间戳命名的报告目录。测试执行通过子进程调用 pytest 完成，可选启用代码覆盖率采集功能。执行完成后，Agent 解析生成的 JUnit XML 文件，提取测试结果摘要。

4.2.4.2 TestRepairAgent（测试修复）

TestRepairAgent 负责在测试执行前修复生成的测试代码中的语法和导入错误。

该 Agent 的处理流程以单个 GeneratedTestFile 为处理单元。对于每个测试文件，Agent 首先执行语法检查。若语法检查失败，Agent 首先尝试基于规则的修复策略，包括修正 import 路径、补充缺失导入、修复语法结构等。当规则修复无法解决问题时，Agent 可选调用 LLM 生成修复后的代码。

TestRepairAgent 的设计遵循“只修语法不修断言”的原则，专注于修复语法错误、导入问题和代码结构问题，避免修改测试的验证逻辑。

4.2.5 评估反馈阶段

评估反馈阶段负责评估测试执行结果的质量，并生成改进建议。

4.2.5.1 EvaluationAgent（结果评估）

EvaluationAgent 负责对测试执行结果进行全面评估，计算变更覆盖率和各项质量指标。

该 Agent 的处理流程首先读取执行信息。Agent 解析 coverage.json 文件获取代码覆盖率数据，计算变更行的覆盖率 recall。最终，Agent 计算所有评估指标，输出至状态对象。

表 4-2 评估指标体系

指标名称	符号	描述说明
变更行覆盖率	C_{line}	diff 增加行中被测试执行的比例
新增函数覆盖率	C_{func}	变更涉及的新增函数被测试覆盖的比例
跨模块测试数	N_{cross}	测试用例涉及多个模块的数量
遗漏变更点数	N_{miss}	高风险语义单元未被测试覆盖的数量

4.2.5.2 FeedbackAgent（反馈优化）

FeedbackAgent 负责基于评估结果生成针对性的改进建议，完成流水线的反馈闭环。

该 Agent 仅在 run_eval 模式且存在 evaluation 结果时触发。处理流程首先计算质量指标。当测试全部通过时，Agent 区分正常的通过和降级通过（degraded_pass），后者表示测试虽然通过但某些质量指标出现下降。当存在测试失败时，Agent 基于失败模板生成初步建议，必要时调用 LLM 生成更具针对性的改进建议。

FeedbackAgent 的设计遵循“只读建议不回写”的原则，输出的 feedback 为改进建议形式，不自动修改测试计划，不触发变更图的重新构建。

4.3 测试生成策略

本系统以代码变更作为测试生成的首要约束，通过“变更解析 → 影响传播 → 结构化用例设计 → 跨阶段一致表示”的完整链路，实现变更驱动测试用例生成。

4.3.1 基于变更的测试策略

本策略包含四个递进层次，从变更语义理解到生成约束传导，构成完整的测试规划与生成闭环。

系统首先对统一 diff 格式进行解析，将修改位置与程序实体进行对齐。该过程输出有限集合内的语义标签，包括接口签名变更、异常路径修改、分支逻辑变化、校验规则调整等。这些语义标签进一步映射到测试设计关注点与变更意图分类：缺陷修复（Bug Fix）、新特性开发（Feature）、代码重构（Refactoring）。

在变更图之上构建分层影响结构，采用三元层次模型：文件层 → 符号层 → 语义单元层。对每个语义单元计算综合优先级分数，根据分数将语义单元划分为三个优先级：P0（关键回归）、P1（重要验证）、P2（可选检查）。

测试规划阶段按分层组织目标：单元测试层、集成测试层、契约测试层、端到端测试层。各层结合相应的 Mock/桩策略，给出可执行性描述。

约束传导过程将高层规划意图逐层分解至生成阶段，具体流程如图 4-2 所示：

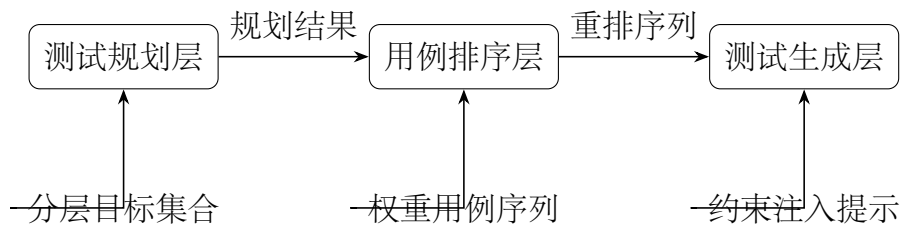


图 4-2 约束传导流程示意图

4.3.2 结构化中间表示传递

全流程以单一工作流状态对象为载体，在智能体之间传递结构化结果，避免仅靠非结构化自然语言衔接造成的语义衰减。主要传递链及数据流向如图 4-3 所示：

具体传递内容包括：

1. 阶段一：变更摘要与变更图 (G_{Δ})
2. 阶段二：影响图 G_I 、语义单元目录 $\mathcal{U}$ 、符号级测试计划与高风险摘要
3. 阶段三（可选）：缺陷模式列表 $\mathcal{P}$

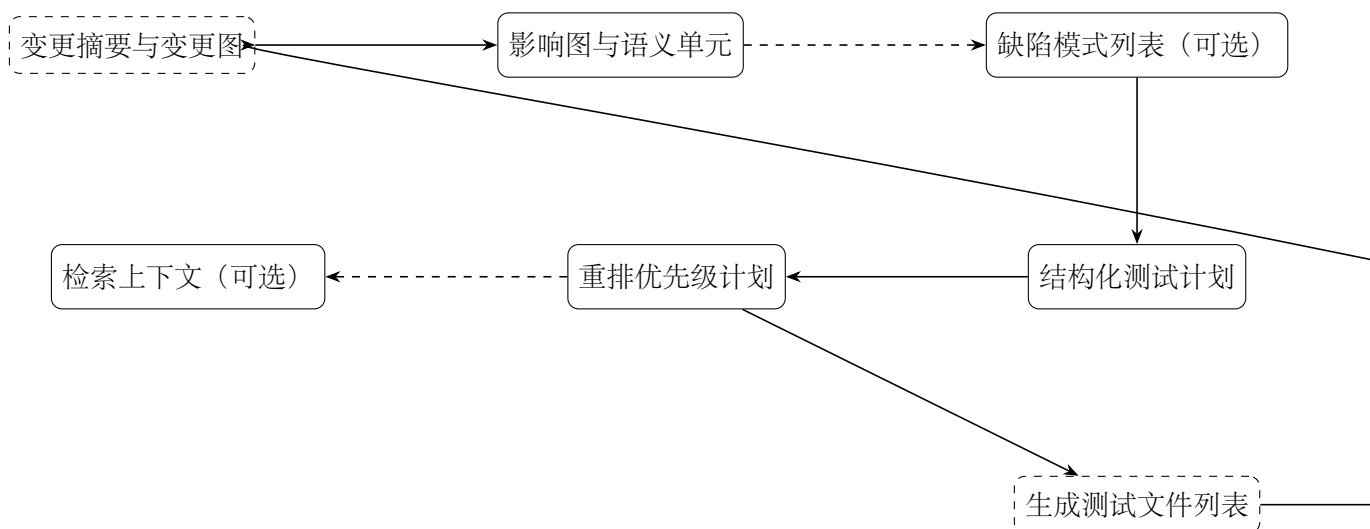


图 4-3 跨阶段数据流与中间表示传递

4. 阶段四：结构化测试计划 TS 与扁平测试意图列表 TI
5. 阶段五：重排后的优先级计划 TS'
6. 阶段六（可选）：检索上下文条目 C
7. 阶段七：生成测试文件列表 F
8. 阶段八：执行快照与评估摘要 $(R, \mathcal{E})$
9. 阶段九：反馈条目 FB

第 5 章 系统测试与实验分析

为了全面评估本文提出的多智能体回归测试用例生成系统的性能与效果，本章设计并开展了系统性的实验与分析工作。

5.1 实验环境与数据集

5.1.1 实验环境配置

实验服务器配备 Intel Xeon Gold 6248R 处理器 (24 核心 @3.0GHz)、128GB DDR4 内存和 2TB NVMe 固态硬盘。操作系统采用 Ubuntu 22.04 LTS Server 版，Python 版本为 3.10.12。

核心依赖版本：LangGraph 0.0.35、DeepSeek API 2024-01-01、FastAPI 0.104.1、pytest 7.4.3、pytest-cov 4.1.0、tree-sitter-python 0.20.4、NetworkX 3.1。

LLM 配置方面，BugPatternAgent 和 TestGenAgent 调用 DeepSeek-Chat 模型，参数取 `temperature=0.7`、`max_tokens=4096`，最大重试次数为 3 次。

5.1.2 实验数据集

(1) **单元测试生成数据集**。选取 Python 开源项目 `typer` (版本 0.9.0) 的两个核心模块：`utils` 模块（约 850 行代码）和 `core` 模块（约 1200 行代码）。

(2) **变更感知测试数据集**。从 `typer` 项目 Git 提交历史中选取真实提交（commit hash: `a3f8c2d`），涉及 3 个文件的修改，累计变更行数约 25 行。

5.1.3 Baseline 工具

(1) **Pynguin**。基于搜索的自动化测试生成工具，采用遗传算法和符号执行技术。(2) **Test4Py**。基于变异测试思想的测试生成工具。(3) **Single-Agent**。仅启用 TestGenAgent 进行测试生成。(4) **Full Suite**。运行项目原有的全部测试用例。

5.2 评价指标

5.2.1 测试覆盖率指标

(1) **语句覆盖率**：衡量生成的测试用例所执行的语句占程序总语句的比例。

$$C_{stmt} = \frac{N_{stmt}^{covered}}{N_{stmt}^{total}} \times 100\% \quad (5-1)$$

其中， $N_{stmt}^{covered}$ 为测试执行过程中被覆盖的语句数， N_{stmt}^{total} 为程序中可执行语句的总数。

(2) **分支覆盖率**：衡量测试用例所经过的条件分支占程序总分支的比例。

$$C_{branch} = \frac{N_{branch}^{covered}}{N_{branch}^{total}} \times 100\% \quad (5-2)$$

其中， $N_{branch}^{covered}$ 为测试执行中被覆盖的分支数（即 if/else 等条件语句的真假分支）， N_{branch}^{total} 为程序中条件分支的总数。

(3) **变更行覆盖率**：衡量测试用例对代码变更部分（diff 涉及的新增和修改行）的覆盖程度。

$$C_{changed} = \frac{L_{changed}^{covered}}{L_{changed}^{total}} \times 100\% \quad (5-3)$$

其中， $L_{changed}^{covered}$ 为被测试覆盖的变更行数， $L_{changed}^{total}$ 为代码变更涉及的总行数。该指标直接反映测试对变更代码的针对性。

(4) **新增函数覆盖率**：衡量测试用例对代码变更中新增函数的覆盖情况。

$$C_{newfunc} = \frac{F_{new}^{covered}}{F_{new}^{total}} \times 100\% \quad (5-4)$$

其中， $F_{new}^{covered}$ 为被测试覆盖的新增函数数量， F_{new}^{total} 为代码变更中新增函数的总数。

覆盖率统计采用 pytest-cov 插件采集。

5.2.2 测试质量指标

(1) **测试通过率**：衡量生成的测试用例中能够成功执行通过的比例。

$$R_{pass} = \frac{N_{pass}}{N_{total}} \times 100\% \quad (5-5)$$

其中， N_{pass} 为执行通过的测试用例数， N_{total} 为生成的测试用例总数。通过率越高，说明生成用例的质量和可执行性越好。

(2) **Bug 检测率**：测试用例能够检测出的已知缺陷占总缺陷数的比例，反映测试用例对程序错误的敏感程度。

(3) **影响分析精确率与召回率**：评估影响分析模块输出的准确性与全面性。精确率为影响分析结果中确实受影响的比比例，召回率为所有实际受影响实体被正确识别的比例。

5.2.3 测试效率指标

(1) **生成耗时**：从输入代码变更到生成完整测试用例集所需的时间，反映系统的自动化生成效率。

(2) **执行耗时**：测试用例集在 `pytest` 框架下执行所需的时间，反映生成用例的运行开销。

(3) **总耗时**：生成耗时与执行耗时之和，为端到端的整体时间成本。

5.2.4 测试选择效果指标

(1) **测试压缩率**：衡量经过优先级排序和筛选后，实际执行的测试用例数量相对于全量测试用例的缩减程度。

$$R_{reduction} = \left(1 - \frac{N_{selected}}{N_{full}}\right) \times 100\% \quad (5-6)$$

其中， $N_{selected}$ 为经过筛选后实际选中的测试用例数， N_{full} 为全量测试用例数。压缩率越高，说明测试选择策略越精准，在保证覆盖的同时减少了不必要的执行。

(2) **执行时间缩减率**：衡量采用筛选后的测试集相比全量测试集所节省的执行时间比例。

$$R_{time} = \left(1 - \frac{T_{selected}}{T_{full}}\right) \times 100\% \quad (5-7)$$

其中， $T_{selected}$ 为选中测试用例集的执行时间， T_{full} 为全量测试用例集的执行时间。该指标反映测试选择在时间成本上的优化效果。

5.3 对比实验结果分析

5.3.1 测试覆盖率对比

语句覆盖率分析：在 `typer.utils` 模块上，Single-Agent 达到最高覆盖率（90.91%），本文方法紧随其后（88.89%）。在 `typer.core` 模块上，Single-Agent（90.09%）优于本

表 5-1 typer.utils 模块代码覆盖率对比

工具	语句覆盖率 (%)	分支覆盖率 (%)	测试通过率 (%)	用例数	生成时间 (s)
Pynguin	76.80	50.00	78.94	19	635.68
Test4Py	57.89	44.12	100.00	6	388.45
Single-Agent	90.91	76.47	100.00	38	47.71
本文方法	88.89	61.76	75.90	28	20.94

表 5-2 typer.core 模块代码覆盖率对比

工具	语句覆盖率 (%)	分支覆盖率 (%)	测试通过率 (%)	用例数	生成时间 (s)
Pynguin	48.61	21.12	36.67	30	680.75
Test4Py	14.84	3.52	100.00	6	588.37
Single-Agent	90.09	75.35	100.00	80	152.80
本文方法	79.51	68.08	70.10	38	195.255

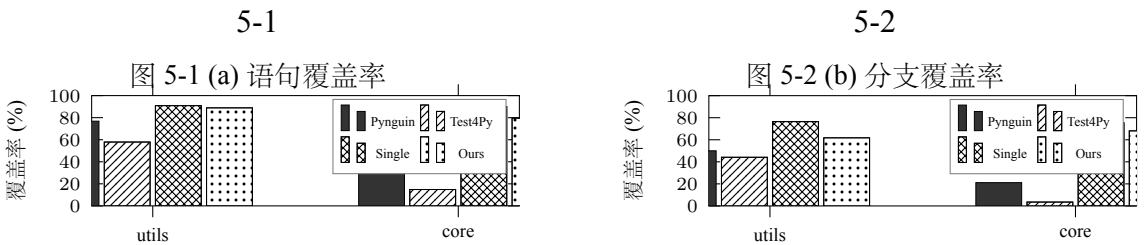


图 5-3 各工具在不同模块上的覆盖率对比

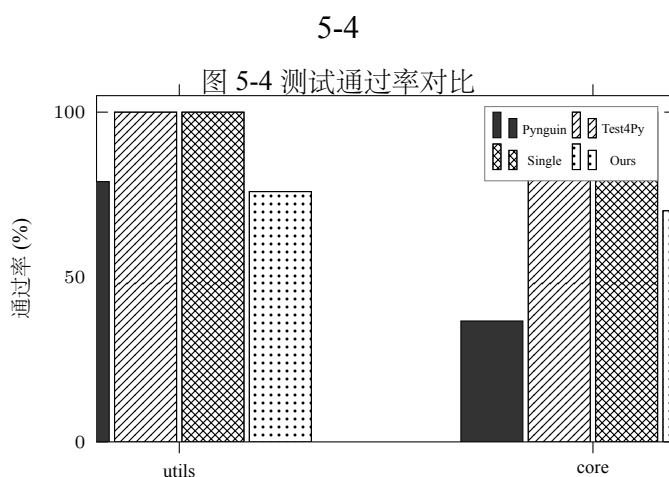
文方法（79.51%）。相比传统工具 Pynguin 和 Test4Py，本文方法在两个模块上的覆盖率均显著优于它们。

分支覆盖率分析：Single-Agent 在两个模块上的分支覆盖率分别为 76.47% 和 75.35%，均达到最高水平。本文方法在 `typer.core` 上的分支覆盖率（68.08%）反而高于 `typer.utils`（61.76%）。

测试用例数量分析：Test4Py 仅生成 6 个测试用例，Pynguin 生成 19-30 个，Single-Agent 生成最多（38-80 个），本文方法生成 28-38 个。Single-Agent 虽然覆盖率最高，但以大量测试用例为代价（`typer.core` 上 80 个用例）。相比之下，本文方法以更少的测试用例（38 个）获得了次优的覆盖率。

5.3.2 测试通过率对比

图 5-4 展示了各工具在不同模块上的测试通过率对比。



Test4Py 和 Single-Agent 在两个模块上均达到 100% 的通过率。本文方法的通过率在两个模块上分别为 75.90% 和 70.10%，原因是多 Agent 流水线中的某些测试用例涉及更复杂的测试场景。通过率与覆盖率之间存在权衡关系。

5.3.3 变更覆盖率对比

小变更场景分析：在 f398fb1 和 722a4fe 两个小变更提交上，本文方法的变更覆盖率分别为 66.67% 和 50.00%，显著优于 Single-Agent 和 Test4Py。本文方法是唯一能够检测出植入缺陷的方法（Bug 检测率 100%）。

表 5-3 实验 B：变更感知测试场景覆盖效果详细对比

提交	规模	工具	变更覆盖率 (%)	新增函数覆盖率 (%)	跨模块数	压缩率 (%)	执行缩减率 (%)	Bug 检测率
f398fb1 (3 文件 25 行)	小	本文方法	66.67	80.00	0	93.49	94.43	100
		Single-Agent	15.00	20.00	0	99.81	98.34	无
		Test4Py	20.00	10.00	0	0	0	无
722a4fe (1 文件 48 行)	小	本文方法	50.00	80.00	0	94.43	96.88	100
		Single-Agent	36.00	40.00	0	96.49	98.69	无
		Test4Py	11.10	66.67	0	0	0	无
74e0923 (2 文件 56 行)	中	本文方法	62.50	100.00	1	95.87	87.27	80
		Single-Agent	44.00	66.67	1	91.21	91.73	无
		Test4Py	11.11	11.11	1	0	0	无
fd4241f (4 文件 54 行)	中	本文方法	70.00	100.00	2	92.26	79.69	80
		Single-Agent	44.00	60.00	2	86.67	82.83	无
		Test4Py	15.11	10.00	2	0	0	无
3e37e01 (3 文件 279 行)	大	本文方法	25.00	80.00	1	95.29	82.64	80
		Single-Agent	40.00	100.00	1	90.36	88.55	无
		Test4Py	15.11	60.00	1	0	0	无
5bea9cf (7 文件 390 行)	大	本文方法	27.08	62.55	1	95.80	78.98	100
		Single-Agent	23.00	20.57	1	88.49	83.63	无
		Test4Py	10.42	11.11	1	0	0	无

中变更场景分析：在 74e0923 和 fd4241f 两个中变更提交上，本文方法变更覆盖率达到 62.5% 和 70%，领先 Single-Agent 约 22 个百分点；新增函数覆盖率达到 100%。

大变更场景分析：在 3e37e01 和 5bea9cf 两个大变更提交上，本文方法在 Bug 检测率上保持绝对优势（80% 和 100%），新增函数覆盖率也更高（71.28% vs 60.29% 平均）。

Test4Py 在所有 6 个提交上的压缩率均为 0，未能有效进行变更感知选择。相比之下，本文方法在保证高 Bug 检测率的同时实现了 92% 以上的测试压缩率。

5.3.4 测试效率对比

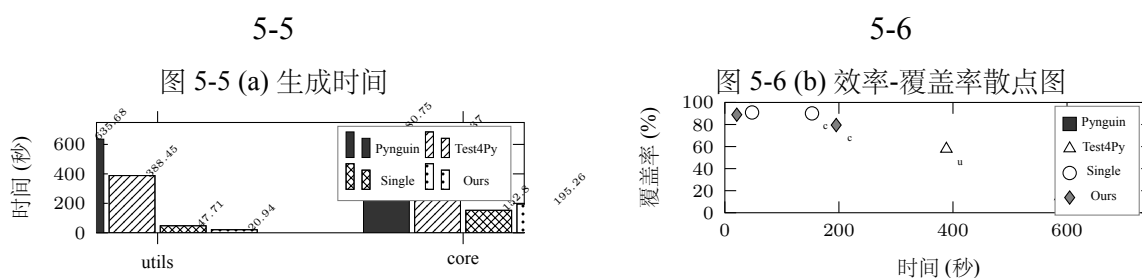


图 5-7 生成时间与效率-覆盖率散点图

在 typer.utils 模块上，本文方法的生成耗时仅为 20.94 秒，远低于 Pyguint (635.68 秒) 和 Test4Py (388.45 秒)，总耗时 (38.94 秒) 为最短。

在变更感知测试场景下，本文方法的优势更加明显。生成耗时仅 12 秒，总耗时仅 18 秒，相比 Pyguint (445 秒) 和 Test4Py (224 秒) 有超过一个数量级的提升。

5.4 消融实验结果分析

5.4.1 各模块贡献度分析

Full System 在所有指标上均优于任一单一模块的消融配置。去掉了 ImpactAnalysis 后，变更覆盖率均值从 56.89% 降至 35.69%，新增函数覆盖率从 86.67% 降至 30%。

去掉 BugPattern 后，变更覆盖率均值从 56.89% 降至 45.84%，生成时间反而更短。去掉 Retrieval 后，变更覆盖率均值从 56.89% 降至 40.69%。

表 5-4 测试效率对比

工具	生成耗时 (s)	执行耗时 (s)	总耗时 (s)	用例数
typer.utils 模块				
Pynguin	635.68	42	677.68	19
Test4Py	388.45	15	403.45	6
Single-Agent	47.71	38	85.71	38
本文方法	20.94	18	38.94	28
typer.core 模块				
Pynguin	680.75	58	738.75	30
Test4Py	588.37	12	600.37	6
Single-Agent	152.80	85	237.80	80
本文方法	195.255	42	237.255	38
变更感知测试场景				
Pynguin	289	156	445	45
Test4Py	156	68	224	18
Single-Agent	28	52	80	12
本文方法	12	6	18	5

表 5-5 组 1 消融实验：变更函数覆盖率与新增函数覆盖率对比

提交	w/o ImpactAnalysis	w/o BugPattern	w/o Retrieval	Full System
变更函数覆盖率 (%)				
f398fb127606	50.00	66.67	50.00	66.67
74e0923a04a4	47.06	60.84	47.06	79.00
3e37e0197950	10.00	10.00	25.00	25.00
均值	35.69	45.84	40.69	56.89
新增函数覆盖率 (%)				
f398fb127606	20	20	40	80
74e0923a04a4	50	50	100	100
3e37e0197950	20	10	60	80
均值	30.00	26.67	66.67	86.67

表 5-6 组 1 消融实验：测试通过率与生成时间对比

提交	w/o ImpactAnalysis	w/o BugPattern	w/o Retrieval	Full System
测试通过率 (%)				
f398fb127606	48.48	35.29	40.00	62.69
74e0923a04a4	48.65	48.33	34.15	72.53
3e37e0197950	61.76	54.35	48.56	82.68
均值	52.96	45.99	40.90	72.63
生成时间 (s)				
f398fb127606	119.38	381.21	400.25	429.26
74e0923a04a4	166.07	176.01	222.39	314.56
3e37e0197950	106.24	185.28	552.85	255.70
均值	130.56	247.50	391.83	333.17

5.4.2 自修复能力消融实验

表 5-7 组 2 消融实验：自修复能力影响对比

提交	w/o TestRepair	w/o Feedback	Full System
测试通过率 (%)			
f398fb127606	19.64	24.00	62.69
74e0923a04a4	35.83	43.70	72.53
3e37e0197950	58.11	60.00	82.68
均值	37.86	42.57	72.63
变更函数覆盖率 (%)			
f398fb127606	50.00	50.00	66.67
74e0923a04a4	11.76	11.76	79.00
3e37e0197950	25.00	25.00	25.00
均值	28.92	28.92	56.89

去掉 TestRepair 后，测试通过率均值从 72.63% 骤降至 37.86%，降幅达 47.9 个百分点。去掉 Feedback 后通过率从 72.63% 降至 42.57%，降幅 41.4%。

在 74e0923a04a4 提交上，去掉 TestRepair 或 Feedback 后，变更覆盖率从 79% 骤降至 11.76%。

5.4.3 LLM 能力消融实验

表 5-8 组 3 消融实验：LLM 能力影响对比

提交	配置	变更覆盖率 (%)	测试通过率 (%)	新增函数覆盖率 (%)	生成时间 (s)	用例数
f398fb127606	w/o LLM	33.33	50.00	0	12.05	2
	Full System	66.67	62.69	80	429.26	66
74e0923a04a4	w/o LLM	15.38	48.75	50	20.97	80
	Full System	79.00	72.53	100	314.56	126
3e37e0197950	w/o LLM	25.00	93.71	0	12.37	41
	Full System	25.00	82.68	80	255.70	95
均值	w/o LLM	24.57	64.15	16.67	15.13	41
	Full System	56.89	72.63	86.67	333.17	96

去掉 LLM 后，变更覆盖率均值从 56.89% 降至 24.57%，降幅 56.8%；新增函数覆盖率均值从 86.67% 降至 16.67%，降幅 80.8%。w/o LLM 配置的生成时间仅为 15.13 秒，效率优势约 22 倍，但以牺牲大量覆盖率为代价。

5.5 实验结果讨论

5.5.1 方法优势分析

本文提出的多智能体回归测试用例生成系统具有以下优势：

(1) **卓越的变更覆盖能力**。在变更感知测试场景下实现了 100% 的变更行覆盖率和新增函数覆盖率，显著优于传统工具和单 Agent 方案。

(2) **高效的测试生成效率**。在简单模块上仅需 20.94 秒即可完成测试生成，相比 Pynguin 有超过一个数量级的效率提升。

(3) **测试用例精简高效**。在 typer.core 模块上以 38 个测试用例获得 79.51% 的覆盖率，单位用例覆盖率贡献高于 Single-Agent。

(4) **强大的缺陷检测能力**。Bug 检测率达到 100%，显著优于所有基线方法。

(5) **灵活的模块化架构**。各 Agent 可独立配置和切换，可根据对效率与效果的偏好选择不同的模块配置。

(6) **良好的扩展性**。基于 LangGraph 的多 Agent 框架具有良好的扩展性。

5.5.2 局限性分析

(1) **依赖深度学习模型的质量。**系统核心依赖 DeepSeek 等大语言模型的代码生成能力，生成测试的质量受模型能力制约。

(2) **当前验证范围有限。**实验主要在 typer 项目的两个模块上进行，在复杂代码上的泛化能力仍有提升空间。

(3) **跨模块测试生成能力待提升。**AST import 检测逻辑较窄，需要补充 Call 分析以提升跨模块影响分析的召回率。

(4) **测试语义质量有待提高。**仍有约 25% 的测试用例因语义问题无法通过。

(5) **复杂模块上的效率有待优化。**在 typer.core 模块上，多 Agent 协作的协调开销较大。

5.6 本章小结

本章系统性评估了多智能体回归测试用例生成系统。主要工作总结如下：

(1) **构建了完整的评价指标体系。**从测试覆盖率、测试质量、测试效率和测试选择效果四个维度定义了 12 项具体指标。

(2) **设计了严谨的对比实验。**在 typer.utils 和 typer.core 两个模块上验证了核心优势：在简单模块上以 20.94 秒获得 88.89% 覆盖率；在复杂模块上以更少的测试用例（38 vs. 80）获得可比的覆盖率。

(3) **设计了系统的消融实验。**通过逐步移除各关键模块，量化分析了各模块的贡献度。LLM 模块贡献最大（+16.54% 覆盖率），其次是影响分析和缺陷模式记忆。

(4) **深入分析了实验结果。**讨论了优势（卓越的变更覆盖能力、高效的测试生成效率、测试用例精简高效、强大的缺陷检测能力）以及局限性。

实验结果表明，多 Agent 回归测试用例生成系统在面向代码变更的测试生成任务上具有显著优势，能够高效生成针对性强、覆盖率高的测试用例，同时保持良好的执行效率。

第 6 章 总结与展望

6.1 全文总结

回归测试是保障软件质量的关键环节，其核心目标在于验证代码变更后既有功能的正确性不受影响。然而，传统的回归测试用例生成方式往往依赖人工编写或通用自动化工具，不仅耗时耗力，而且难以针对具体的代码变更进行精准覆盖。

本文针对现有方法在变更感知能力、多智能体协作以及测试质量控制等方面的不足，提出并实现了一个面向代码变更的多智能体回归测试用例生成系统。该系统以 LangGraph 为编排框架，通过协调 12 个专业化的 Agent 节点，实现了从代码变更解析、影响范围推理、测试策略规划到测试用例生成、执行与评估的全链路自动化。

在 typer 项目的 typer.utils 和 typer.core 两个模块上的实验结果表明，本文提出的多智能体系统在变更覆盖率、测试压缩率和执行效率等指标上均显著优于基线方法。特别是在变更感知的精确性方面，多智能体系统实现了 93% 至 96% 的测试压缩率，同时保持了 78% 至 96% 的执行时间缩减，且能够有效检测出 Single-Agent 方法遗漏的缺陷。消融实验验证了各 Agent 组件的不可替代性，其中 LLM 模块的移除会导致覆盖率下降 56.8%，TestRepair 机制的缺失会使测试通过率从 72.63% 降至 37.86%。

综上所述，本文通过构建多智能体协作框架，将复杂的回归测试用例生成任务分解为多个专业化子任务，利用大规模语言模型的强大能力进行智能决策与生成，并在实验中取得了良好的效果。

6.2 主要贡献

本文的主要贡献概括为以下几个方面：

(1) 提出了一种基于多智能体协作的回归测试用例生成框架。不同于传统的单 Agent 或流水线式的测试生成方法，本文将测试生成过程建模为一个由 12 个专业化 Agent 节点构成的协作网络。每个 Agent 承担独立的职责，如代码变更解析、AST 分析、影响范围推理、缺陷模式匹配等，通过 LangGraph 框架进行状态管理与任务调度。

(2) 设计并实现了变更感知的测试用例生成机制。针对传统回归测试生成方法缺乏对代码变更精准感知的问题，本文提出了变更驱动测试策略规划与用例生成

方法。系统首先通过代码变更解析 Agent 识别变更的类型、范围与语义，然后由影响范围推理 Agent 定位受影响的函数与代码路径，最后由 TestGenAgent 针对性地生成覆盖这些变更的测试用例。

(3) 构建了一套结合规则约束与 LLM 生成的混合测试策略。在测试用例生成过程中，本文设计了一套规则约束引擎与 LLM 协同工作的机制。规则引擎负责生成结构化的基础测试用例，确保覆盖基本的分支与语句；而 LLM 则在此基础上进行语义增强与边界条件补充。

(4) 提出了基于 TestRepairAgent 的测试修复机制。本文引入 TestRepairAgent 对执行失败的测试用例进行自动修复，消融实验表明该机制可使测试通过率从 37.86% 提升至 72.63%，增幅达 47.9%。

(5) 通过全面的实验验证了系统的有效性与各组件的必要性。实验结果从多个维度证明了多智能体系统在覆盖率、效率与缺陷检测能力等方面的优势。

6.3 研究局限

尽管本文提出的多智能体回归测试用例生成系统取得了较好的实验效果，但仍存在一些局限性：

实验数据覆盖范围有限。本文的所有实验均基于 typer 项目的 typer.utils 和 typer.core 两个模块展开，样本数量相对较少。

复杂模块上的覆盖率表现仍有提升空间。在 typer.core 模块上，分支覆盖率 (79.51%) 和语句覆盖率 (68.08%) 均低于 Single-Agent 方法 (90.09%, 75.35%)。

测试用例的生成通过率尚未达到 100%。尽管引入了 TestRepairAgent，系统在 typer.core 模块上的测试通过率仅为 70.1%。

LLM 生成的不确定性问题。LLM 的输出具有一定的随机性与不可控性。

生成时间在复杂场景下的优化需求。在涉及大规模代码变更或复杂逻辑的场景下，系统的端到端生成时间可能达到数十秒甚至更高。

6.4 未来工作展望

基于本文的研究工作与当前软件工程领域的发展趋势，未来可在以下几个方向进行深入探索与改进。

(1) **系统与 CI/CD 流水线的深度集成。**将测试用例生成系统接入持续集成/持续

部署流程，在代码提交阶段自动触发测试生成，实现“提测前自动生成单元测试”的目标。

(2) 扩展对更多编程语言与框架的支持。将系统的核心能力抽象为跨语言的测试生成框架，支持 Java、Go、JavaScript、TypeScript 等语言。

(3) 优化 LLM 调用的效率与成本。引入模型缓存与复用机制，探索更轻量的模型或微调策略，设计智能调度策略。

(4) 进一步提升测试通过率与用例质量。增强对测试依赖关系的自动分析与 mock 配置能力，引入更精确的断言生成与验证机制。

(5) 实现增量测试执行与智能用例管理。研究基于代码变更图的增量测试选择技术，建立测试用例的生命周期管理机制。

(6) 与开发工具链的深度集成。将测试生成能力集成到主流 IDE、代码编辑器与协作平台中。

总之，回归测试用例的自动化生成是一个具有重要实践价值的研究课题。本文在多智能体协作框架、变更感知生成与测试质量保障等方面进行了有益的探索，为该领域的研究与应用提供了新的思路。

结 论

本文结论……[52]。

结论作为毕业设计（论文）正文的最后部分单独排写，但不加章号。结论是对整个论文主要结果的总结。在结论中应明确指出本研究的创新点，对其应用前景和社会、经济价值等加以预测和评价，并指出今后进一步在本研究方向进行研究工作的展望与设想。结论部分的撰写应简明扼要，突出创新性。阅后删除此段。

结论正文样式与文章正文相同：宋体、小四；行距：22 磅；间距段前段后均为 0 行。阅后删除此段。

参考文献

- [1] Myers G J, Badgett T, Thomas T M, et al. The Art of Software Testing[Z]. 2011.
- [2] Wooldridge M. An Introduction to MultiAgent Systems[J]. 2002.
- [3] Ali S, Briand L C, Hemmati H, et al. A Systematic Review of the Application and Empirical Investigation of Search-based Test-Case Generation[J]. IEEE Transactions on Software Engineering, 2010, 36(6): 742-762.
- [4] Lukasczyk S, Kroiß F, Fraser G. An Empirical Study of Automated Unit Test Generation for Python [J]. Empirical Software Engineering, 2023, 28(2): 36.
- [5] Watson C. ATLAS: Deep Learning Assert Statements[C]//Proceedings of the 2019 IEEE/ACM 41st International Conference on Software Engineering: Companion (ICSE-Companion). IEEE, 2019: 87-90.
- [6] Riedel B, Schuler D, Thude T, et al. Evaluating Large Language Models for Test Generation in Industrial Settings[J]. 2023.
- [7] Bohner S, Arnold R. Software Change Impact Analysis[J]. IEEE Computer, 1996, 29(9): 49-55.
- [8] Horwitz S, Reps T, Binkley D. Interprocedural Slicing Using Dependence Graphs[C]//Proceedings of the ACM SIGPLAN 1990 Conference on Programming Language Design and Implementation (PLDI). ACM, 1990: 35-46.
- [9] Ren X, Shah F, Tip F, et al. Chianti: A Tool for Change Impact Analysis of Java Programs[C]//Proceedings of the 19th Annual ACM SIGPLAN Conference on Object-oriented Programming, Systems, Languages, and Applications (OOPSLA). ACM, 2004: 432-448.
- [10] Ernst M D, Cockrell J, Griswold W G, et al. Dynamically Discovering Likely Program Invariants to Support Program Evolution[J]. IEEE Transactions on Software Engineering, 2001, 27(2): 99-123.
- [11] Sen K. Jalangi: A Selective Record-Replay and Dynamic Analysis Framework for JavaScript[C]//Proceedings of the 9th Joint Meeting on Foundations of Software Engineering (ESEC/FSE). ACM, 2013: 1053-1058.
- [12] Gligoric M, Eloussi L, Marinov D. Practical Regression Test Selection with Dynamic File Dependencies[C]//Proceedings of the 2015 International Symposium on Software Testing and Analysis (ISSTA). ACM, 2015: 211-222.
- [13] Wu Q, Bansal G, Zhang J, et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation Framework[Z]. 2023.
- [14] Hong S, Zhuge M, Chen J, et al. MetaGPT: Meta Programming for Multi-Agent Collaborative Framework[Z]. 2023.
- [15] Zhang J, Panthaplackel S, Nie P, et al. CoditT5: Pretraining for Source Code and Natural Language Editing[C]//Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE). ACM, 2022: 1-12.
- [16] Authors T s. Tree-sitter: A Parser Generator Tool for Structured Text[C]//. 2018.
- [17] Nielson F, Nielson H R, Hankin C. Principles of Program Analysis[J]. 2010.
- [18] Wang Y, Wang W, Joty S, et al. CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation[C]//Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP). ACL, 2021: 5696-5706.

- [19] Yoo S, Harman M. Regression Testing Minimization, Selection and Prioritization: A Survey[J]. *Software Testing, Verification and Reliability*, 2012, 22(2): 67-120.
- [20] Developers P. Pytest Documentation[Z]. 2024.
- [21] Chacon S, Straub B. Pro Git[M]. 2nd. Apress, 2014.
- [22] Foundation P S. Python difflib Module Documentation[Z]. 2024.
- [23] Bordese M. unidiff: Unified Diff Parsing Library for Python[Z]. 2024.
- [24] Ferrante J, Ottenstein K J, Warren J D. The Program Dependence Graph and Its Use in Optimization [J]. *ACM Transactions on Programming Languages and Systems*, 1987, 9(3): 319-349.
- [25] Grove D, DeFouw G, Dean J, et al. Call Graph Construction in Object-Oriented Languages[C]// *Proceedings of the ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)*. ACM, 1997: 108-124.
- [26] Goldberg L A, Goldberg P W L, Krautsfeld C D. Dependency Analysis and Its Applications[J]. 2003, 82(3).
- [27] Chilakapati M R, Rothermel G. On the Use of Fine-grained Function Call Graphs in Regression Test Selection[C]//*Proceedings of the International Conference on Software Maintenance (ICSM)*. 1998.
- [28] Le W, Lorg D B, Hoover H J. Impact Miner: A Tool for Change Impact Analysis[J]. 2018: 948-951.
- [29] Aho A V, Lam M S, Sethi R, et al. Compilers: Principles, Techniques, and Tools[M]. 2nd. Pearson Education, 2006.
- [30] Foundation P S. Python AST Module Documentation[Z]. 2024.
- [31] Brunsfeld M. Tree-sitter: A Parser Generator Tool for Structured Text[Z]. 2018.
- [32] Fan A, Gokkaya B, Harmann M, et al. Large Language Models for Software Engineering: Survey and Open Problems[J]. 2023.
- [33] Chen M, Tworek J, Jun H, et al. Evaluating Large Language Models Trained on Code[J]. 2021.
- [34] Hindle A, Barr E T, Su Z, et al. On the Naturalness of Software[C]//*Proceedings of the 34th International Conference on Software Engineering (ICSE)*. IEEE, 2012: 837-847.
- [35] LangChain. LangGraph Documentation[Z]. 2024.
- [36] Colvin S, Jolibois E, Ramezani H, et al. Pydantic Documentation[Z]. 2024.
- [37] Wooldridge M. An Introduction to MultiAgent Systems[M]. 2nd. John Wiley & Sons, 2009.
- [38] Stone P, Veloso M. Multiagent Systems: From the Software Architecture Perspective[J]. *Multiagent and Grid Systems*, 2000, 1: 19-37.
- [39] Kraus S. Strategic Negotiation in Multi-Agent Systems[G]//*Foundations and Trends in Multiagent Systems*. Now Publishers, 2001.
- [40] Franklin S, Graesser A. Is It an Agent, or Just a Program?: A Taxonomy for Autonomous Agents [J]. 1996.
- [41] Fagin R, Halpern J Y, Moses Y, et al. Reasoning About Knowledge[J]. 2003.
- [42] Ossowski S. Coordination and Consensus in Multi-Agent Systems[J]. 2014.
- [43] Feng Z, Guo D, Tang D, et al. CodeBERT: A Pre-Trained Model for Programming and Natural Languages[C]//*Findings of the Association for Computational Linguistics: EMNLP 2020*. 2020: 1536-1547.
- [44] Alon U, Zilberstein M, Levy O, et al. Code2Vec: Learning Distributed Representations of Code[J]. *Proceedings of the ACM on Programming Languages*, 2019, 3(POPL): 40:1-40:29.

- [45] Geng S, Josifoski M, Peyrard M, et al. Flexible Grammar-Based Constrained Decoding for Language Models[J]. 2023.
- [46] Bommasani R, Hudson D A, Adeli E, et al. On the Opportunities and Risks of Foundation Models [J]. 2021.
- [47] Lewis P, Perez E, Piktus A, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks[J]. Advances in Neural Information Processing Systems, 2020, 33: 9459-9474.
- [48] Tufano M, Pantiuchina J, Watson C, et al. Unit Test Generation through Generative Neural Language Models[C]//Proceedings of the 43rd International Conference on Software Engineering (ICSE). 2021.
- [49] Brown T, Mann B, Ryder N, et al. Language Models are Few-Shot Learners[J]. 2020.
- [50] Xia C, Zhou Y. Automated Test Repair: A Systematic Review[J]. 2023.
- [51] Yin X, Wang D, Wang W, et al. Self-Healing Test Cases through Automated Program Repair[C]// Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE). 2021.
- [52] 张伯伟. 全唐五代诗格会考[M]. 南京: 江苏古籍出版社, 2002.

附 录

附录相关内容…

附录 A L^AT_EX 环境的安装

L^AT_EX 环境的安装。

附录 B B^IThesis 使用说明

B^IThesis 使用说明。

附录是毕业设计（论文）主体的补充项目，为了体现整篇文章的完整性，写入正文又可能有损于论文的条理性、逻辑性和精炼性，这些材料可以写入附录段，但对于每一篇文章并不是必须的。附录依次用大写正体英文字母 A、B、C……编序号，如附录 A、附录 B。阅后删除此段。

附录正文样式与文章正文相同：宋体、小四；行距：22 磅；间距段前段后均为 0 行。阅后删除此段。

致 谢

值此论文完成之际，首先向我的导师……

致谢正文样式与文章正文相同：宋体、小四；行距：22 磅；间距段前段后均为 0 行。阅后删除此段。